A Major Project Report On

BLOCKCHAIN INTEGRATION WITH CLOUD STORAGE FOR SECURE AND TRANSPARENT FILE MANAGEMENT

Project submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

BY

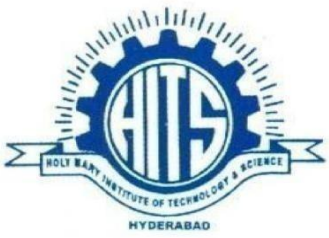
T. SRIYA REDDY	**(22C91A05H1)**
M.SOHAN KRISHNA	**(22C91A05C9)**
S. VISWANATH	**(22C91A05F9)**
Y. SNEHA DEEPIKA	**(22C91A05I8)**

Under the Esteemed guidance of

Dr. M SARAVANAN

Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



HOLY MARY INSTITUTE OF TECHNOLOGY & SCIENCE

(COLLEGE OF ENGINEERING)

(Approved by AICTE New Delhi, Permanently Affiliated to JNTU Hyderabad, Accredited by NAAC with 'A' Grade) **Bogaram (V), Keesara (M), Medchal District -501 301.**

2025 – 2026

HOLY MARY INSTITUTE OF TECHNOLOGY & SCIENCE

(COLLEGE OF ENGINEERING)

(Approved by AICTE New Delhi, Permanently Affiliated to JNTU Hyderabad, Accredited by NAAC with 'A' Grade) **Bogaram (V), Keesara (M), Medchal Dist-501301.**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the major project entitled "BLOCKCHAIN INTEGRATION WITH CLOUD STORAGE FOR SECURE AND TRANSPARENT FILE MANAGEMENT" is being submitted by T. SRIYA REDDY (22C91A05H1), M. SOHAN KRISHNA (22C91A05C9), S. VISWANATH (22C91A05F9), and Y. SNEHA DEEPIKA (22C91A05I8) in Partial fulfilment of the academic requirements for the award of the degree of Bachelor of Technology in "COMPUTER SCIENCE AND ENGINEERING" HOLY MARY INSTITUTE OF TECHNOLOGY & SCIENCE, JNTU HYDERABAD during the year 2022-2026

INTERNAL GUIDE

Dr. M. Saravanan

Professor

Dept. of Computer Science & Engineering

HEAD OF THE DEPARTMENT

Mr. A. Jitendra M Tech (Ph.D.)

Associate Professor

Dept. of Computer Science& Engineering

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of the people who made it possible, who's constant guidance and encouragement crown all effort with success.

We take this opportunity to express my profound gratitude and deep regards to our Guide **Dr. M. Sarvanan, Professor,** Dept. of Computer Science & Engineering, Holy Mary Institute of Technology & Science for his / her exemplary guidance, monitoring and constant encouragement throughout the project work.

Our special appreciation to our supervisor's **Dr. Prasad Dharnasi and Dr. M. Saravanan Professor's of Computer Science & Engineering Department,** Holy Mary Institute of Technology & Science who has given immense support throughout the course of the project.

Our special thanks to **Mr. Jitendra Alaparthi, Head of the Department**, Dept. of Computer Science & Engineering, Holy Mary Institute of Technology & Science who has given an immense support throughout the course of the project.

We also thank to **Dr. P. Chandra Shekar,** the **honorable Principal** of our college Holy Mary Institute of Technology & Science for providing me the opportunity to carry out this work.

At the outset, we express our deep sense of gratitude to the beloved **Chairman Dr**. **Arminda Siddharth Reddy & Secretary Dr. A. Vijaya Sarada Reddy** of **Holy Mary Institute of Technology & Science**, for giving us the opportunity to complete our course of work.

We are obliged to **staff members** of Holy Mary Institute of Technology & Science for the valuable information provided by them in their respective fields. We are grateful for their cooperation during the period of my assignment.

Last but not the least we thank our **Parents**, and **Friends** for their constant encouragement without which this assignment would not be possible.

T. SRIYA REDDY	(22C91A05H1)
M. SOHAN KRISHNA	(22C91A05C9)
S. VISWANATH	(22C91A05F9)
Y. SNEHA DEEPIKA	(22C91A05I8)

DECLARATION

This is to certify that the work reported in the present project titled **"BLOCKCHAIN INTEGRATION WITH CLOUD STORAGE FOR SECURE AND TRANSPARENT FILE MANAGEMENT"** is a record of work done by us in the Department of Computer Science & Engineering, Holy Mary Institute of Technology and Science.

To the best of our knowledge, no part of the thesis is copied from books/journals/internet and wherever the portion is taken, the same has been duly referred to in the text. The reports are based on the project work done entirely by us not copied from any other source.

T. SRIYA REDDY (22C91A05H1)
M. SOHAN KRISHNA (22C91A05C9)
S. VISWANATH (22C91A05F9)
Y. SNEHA DEEPIKA (22C91A05I8)

CONTENTS

LIST OF FIGURES

LIST OF IMAGES

LIST OF TABLES

LIST OF ABBREVIATIONS

BC	Blockchain
CS	Cloud Storage
IPFS	Interplanetary File System
AL	Artificial Intelligence
ML	Machine Learning
API	Application Programming Interface
SC	Smart Contract
ETH	Ethereum
DLT	Distributed Ledger Technology
SHA	Secure Hash Algorithm
AES	Advanced Encryption Standard
DB	Database
SQL	Structured Query Language
UI	User Interface
IDE	Integrated Development Environment
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
UAT	User Acceptance Testing
DFD	Data Flow Diagram
UML	Unified Modelling Language
KPI	Key Performance Indicator

ABSTRACT

The rapid growth of digital data and cloud computing has significantly increased the need for secure, reliable, and transparent file storage systems. Traditional cloud storage solutions rely on centralized architectures, which are vulnerable to data breaches, unauthorized access, and lack of transparency. These limitations raise serious concerns regarding data integrity and user trust.

This project presents the design and implementation of a secure file management system by integrating blockchain technology with cloud storage. The proposed system uses cryptographic techniques to encrypt files before storing them in cloud environments such as IPFS or cloud servers. A unique hash is generated for each file and stored on the blockchain, ensuring data integrity and preventing unauthorized modifications.

Smart contracts are utilized to manage file ownership, access control, and transaction records in a decentralized and transparent manner. During file retrieval, the system verifies data integrity by comparing the current file hash with the stored blockchain hash, ensuring that the file has not been tampered with.

The system is implemented using modern technologies such as Node.js/Python, Solidity, Web3.js, and decentralized storage mechanisms. It provides secure file upload, storage, sharing, and verification features while maintaining transparency and trust among users.

Overall, the proposed solution offers a scalable, efficient, and tamper-proof approach to cloud-based file management, making it suitable for applications requiring high levels of security, integrity, and accountability.

KEYWORDS

Blockchain, Cloud storage, Secure file management, Data integrity, Privacy, Smart contracts, Decentralized System, Access Control, Hash Verification, Secure File Management.

1. INTRODUCTION

Blockchain Integration with Cloud Storage for Secure and Transparent File Management is a secure file storage system designed to enhance the reliability, transparency, and security of cloud-based file management. Traditional cloud storage systems often face challenges such as unauthorized access, data tampering, lack of transparency, and limited traceability. This project addresses these issues by integrating blockchain technology, encryption, and decentralized storage to create a highly secure and verifiable file management platform. The system allows users to upload, store, and retrieve files through a web-based interface. When a user uploads a file, the system first encrypts the file using AES-256 encryption to ensure confidentiality. The encrypted file is then stored in IPFS (Interplanetary File System), a decentralized storage network that provides distributed and reliable data storage. IPFS generates a unique content-based hash for each uploaded file. To guarantee data integrity and transparency, the generated file hash is recorded on the Ethereum blockchain using a smart contract. Since blockchain records are immutable and tamper-proof, any modification to the stored file can be detected by comparing the stored blockchain hash with the hash of the retrieved file. This ensures that users can verify the authenticity and integrity of their files at any time. The system also includes strong authentication and access control mechanisms. Users must register and log in using a secure authentication process that includes password verification and OTP-based multi-factor authentication. Additionally, JWT-based session management and role-based access control (RBAC) are implemented to restrict unauthorized access and ensure that users can only interact with files they are permitted to access. The architecture of the system consists of several integrated components. The frontend interface is developed using HTML, CSS, and JavaScript, allowing users to interact with the platform through a browser. The backend server, built with Node.js and Express, handles authentication, file processing, encryption, and communication with the blockchain and storage systems. MongoDB is used for storing user data and metadata related to files and transactions. Smart contracts deployed on the Ethereum blockchain using Hardhat maintain the immutable record of file hashes. By combining cloud storage, blockchain verification, encryption, and decentralized storage, this project creates a secure and transparent file management system.

1.1 OVERVIEW

"Blockchain Integration with Cloud Storage for Secure and Transparent File Management" represents an advanced approach to modern data storage by combining the capabilities of cloud computing with the security features of blockchain technology. In today's digital era, organizations and individuals generate and store massive amounts of data, ranging from personal files to sensitive business information. Cloud storage has become the preferred solution due to its scalability, accessibility, and cost-effectiveness. However, traditional cloud systems are centralized, which introduces several risks such as data breaches, unauthorized access, and lack of transparency.

Centralized storage systems rely on a single authority or service provider to manage data. This creates a dependency on third parties and increases the risk of system failures, insider threats, and cyber-attacks. Users often have limited control over their data and cannot verify whether their files have been altered or accessed without permission. These limitations highlight the need for a more secure and trustworthy system.

The proposed system addresses these challenges by integrating blockchain technology with cloud storage. Blockchain is a decentralized and distributed ledger technology that ensures data immutability, transparency, and security. In this system, files uploaded by users are first encrypted using advanced cryptographic techniques to ensure confidentiality. The encrypted files are then stored in decentralized storage systems such as IPFS or traditional cloud servers. A unique cryptographic hash is generated for each file, which acts as a digital fingerprint. This hash is stored on the blockchain using smart contracts.

A unique cryptographic hash is generated for each file, which acts as a digital fingerprint. This hash is stored on the blockchain using smart contracts, ensuring that any modification to the file can be easily detected. The use of blockchain eliminates the need for a central authority and provides a transparent mechanism for tracking file transactions.

1.2 PROBLEM STATEMENT

The rapid adoption of cloud storage solutions has transformed the way data is stored and accessed. However, despite its advantages, traditional cloud storage systems suffer from several critical limitations that pose significant risks to data security, integrity, and user trust. These systems are primarily based on centralized architectures, where data is stored and managed by a single service provider. This centralization introduces vulnerabilities such as data breaches, unauthorized access, and system failures.

One of the major concerns with centralized cloud storage is the risk of data tampering. Since the service provider has full control over the stored data, users cannot independently verify whether their data has been modified or accessed without authorization. This lack of transparency creates trust issues between users and service providers.

Another significant issue is the presence of a single point of failure. If the central server experiences downtime due to technical failures, cyber-attacks, or maintenance issues, users may lose access to their data. In extreme cases, data loss can occur, leading to serious consequences for businesses and individuals.

Additionally, insider threats and misconfigurations can compromise data security. Employees or administrators with privileged access may intentionally or unintentionally modify or leak sensitive data. Traditional systems also lack robust mechanisms for verifying data integrity, making it difficult to detect unauthorized changes.

To overcome these challenges, there is a need for a decentralized, secure, and transparent file management system that ensures data integrity and provides verifiable proof of authenticity.

1.3 OBJECTIVE

The primary objective of the project “Blockchain Integration with Cloud Storage for Secure and Transparent File Management” is to design and develop a secure, decentralized, and efficient system for storing and managing digital files while ensuring data integrity, confidentiality, and transparency.

One of the key objectives is to enhance data security by implementing encryption techniques such as AES to protect files before storing them in cloud storage. This ensures that even if unauthorized access occurs, the data remains secure and unreadable. The system also aims to prevent data tampering by generating a unique cryptographic hash (using SHA-256) for each file and storing it on the blockchain.

Another important objective is to ensure transparency and trust by leveraging blockchain technology. By storing file hashes and transaction details on an immutable ledger, the system enables users to verify the authenticity of their data at any time. This eliminates reliance on third-party service providers and builds user confidence.

The project also focuses on implementing secure user authentication and access control mechanisms. Only authorized users are allowed to upload, access, or download files, ensuring data privacy and protection. Technologies such as JWT (JSON Web Tokens) are used to manage secure user sessions.

Additionally, the system aims to provide efficient and scalable file management by integrating decentralized storage platforms like IPFS. This ensures high availability, fault tolerance, and efficient data retrieval.

Another objective is to create a user-friendly interface that allows easy interaction with the system, making it accessible to both technical and non-technical users. The system is designed to be scalable and adaptable, allowing future enhancements such as AI-based security, multi-cloud integration, and real-time monitoring.

Overall, the objective of this project is to develop a robust, secure, and transparent file management system that overcomes the limitations of traditional cloud storage and provides a reliable solution for modern digital environments.

2. LITERATURE REVIEW (2017 – 2026)

Blockchain-Based Security Architecture for Distributed Cloud Storage by Jiaxing Li et al 2017 [1]. With the development of ICT industry, the volume of produced data is experiencing tremendous growth, which motivates more demands of storage capacity. Because of the limited storage capacity of users' terminals, more applications prefer to upload data to cloud platforms. However, it is well known that security should not be neglected in existing cloud storage architectures. Motivated by the increasing popularity of emerging blockchain technology, we propose a blockchain-based security architecture for distributed cloud storage. Numerical experimental results show that the proposed architecture outperforms the traditional cloud storage architectures in terms of security, with acceptable network transmission delay.

Decentralized Cloud Storage Using Blockchain by Meet Shah et al 2020 [2]. Cloud storage is one of the leading options to store massive data, however, the centralized storage approach of cloud computing is not secure. On the other hand, Blockchain is a decentralized cloud storage system that ensures data security. Any computing node connected to the internet can join and form peers' network thereby maximizing resource utilization. Blockchain is a distributed peer to peer system where each node in the network stores a copy of blockchain thus, making it immutable. In the proposed system, the user's file is encrypted and stored across multiple peers in the network using the IPFS (Interplanetary File System) protocol. IPFS creates hash value. The hash value indicates the path of the file and is stored in the blockchain. This paper focuses on decentralized secure data storage, high availability of data, and efficient utilization of storage resources.

Blockchain Based Cloud Computing: Architecture and Research Challenges by Ch V N U Bharathi Murthy et al 2022 [3]. Blockchain technology is a distributed ledger with records of data containing all details of the transactions carried out and distributed among the nodes present in the network. All the transactions carried out in the system are confirmed by consensus mechanisms, and the data once stored cannot be altered. Blockchain technology is the necessary technology behind Bitcoin, which is a popular digital Cryptocurrency.

Blockchain-Based Fair Payment Smart Contract for Public Cloud Storage Auditing by H. Wang et al 2024 [4]. Cloud storage plays an important role in today's cloud ecosystem. Increasingly clients tend to outsource their data to the cloud. Despite its copious advantages, integrity has always been a significant issue. The audit method is commonly used to ensure integrity in cloud scenarios. However, traditional auditing schemes expect a third-party auditor (TPA), which is not always available in the real world. Also, the former scheme implies a limited pay-as-you-go service, as it requires the client to pay for the service in advance.

Blockchain Integration in Critical Systems Enhancing Transparency, Efficiency, and Real-Time Data Security in Agile Project Management, Decentralized Finance (DeFi), and Cold Chain Management by Odunayo Akindotei et al 2025 [5]. Blockchain technology improves transparency, security, and efficiency in critical systems. It is applied in Agile Project Management, Decentralized Finance (DeFi), and Cold Chain Management using decentralized ledgers and smart contracts. In Agile projects, it enhances collaboration and accountability. In DeFi, it secures digital transactions and reduces fraud. In Cold Chain Management, it ensures traceability and protects temperature-sensitive goods while preventing data tampering.

Integrated Blockchain and Cloud Computing Systems a Systematic Survey, Solutions, and Challenges by Neeraj Kumar et al 2026 [6]. Cloud computing is a network model of on demand access for sharing configurable computing resource pools. Compared with conventional service architectures, cloud computing introduces new security challenges in secure service management and control, privacy protection, data integrity protection in distributed databases, data backup, and synchronization. Blockchain can be leveraged to address these challenges, partly due to the underlying characteristics such as transparency, traceability, decentralization, security, immutability, and automation. We present a comprehensive survey of how blockchain is applied to provide security services in the cloud computing model and we analyse the research trends of blockchain-related techniques in current cloud computing models.

2.1 PROJECT REVIEW

Cloud storage has become a fundamental component of modern computing, allowing users to store and access data remotely without the need for physical storage devices. However, most cloud storage solutions operate on centralized architectures, where data is stored on servers managed by third-party service providers. While this approach offers scalability and convenience, it introduces several challenges. Centralized systems are prone to single points of failure, meaning that if the server is compromised or goes offline, users may lose access to their data. Additionally, users must trust service providers to maintain the integrity and confidentiality of their data, which may not always be guaranteed.

To address these issues, this project incorporates blockchain technology, which provides a decentralized and immutable ledger for recording transactions. Blockchain eliminates the need for a central authority by distributing data across multiple nodes, ensuring that no single entity has complete control over the system. This decentralized approach enhances security and reduces the risk of data manipulation or unauthorized access. By combining blockchain with cloud storage, the system ensures that data is stored securely while maintaining transparency and verifiability.

The proposed system uses Interplanetary File System (IPFS) as a decentralized storage solution. IPFS is a distributed file system that allows files to be stored across multiple nodes, ensuring high availability and fault tolerance. When a file is uploaded to IPFS, it is assigned a unique content identifier (CID), which is generated based on the file's content. This ensures that any change to the file will result in a different CID, making it easy to detect tampering.

In the proposed system, when a user uploads a file, the file is first encrypted using cryptographic techniques to ensure confidentiality. The encrypted file is then stored in IPFS or cloud storage, and a unique hash of the file is generated using algorithms such as SHA-256. This hash acts as a digital fingerprint of the file and is stored on the blockchain using smart contracts. The use of blockchain ensures that the hash cannot be altered, providing a reliable mechanism for verifying file integrity.

2.1.1 BUILDING THE SYSTEM INFRASTRUCTURE

1. System Architecture Design:

The system uses a client–server architecture integrated with blockchain to provide secure and scalable file management. The frontend layer (HTML, CSS, JavaScript, or React) allows users to register, log in, upload, and download files. The backend layer (Node.js or Fast API) manages application logic, authentication, and communication between system components. The blockchain layer (Ethereum and smart contracts) stores immutable file hashes to ensure file integrity. Encrypted files are stored in cloud or decentralized storage such as IPFS, AWS, or Firebase, improving security, scalability, and maintainability.

2. Frontend Infrastructure:

The frontend infrastructure provides an interactive interface that allows users to access and manage the application. It enables users to register, log in, upload files, view file history, and verify file integrity easily without needing technical knowledge. The frontend communicates with the backend server through APIs to process user requests and display results. It is developed using HTML, CSS, and JavaScript, and may also use frameworks like React to improve performance and user experience.

3. Backend Infrastructure:

The backend infrastructure manages the core functionality of the system, including user authentication, file processing, blockchain interaction, and API communication. It processes user requests from the frontend, verifies credentials, and ensures secure handling of files. The backend also communicates with the blockchain network to store and retrieve file hashes and interacts with cloud or decentralized storage (like IPFS) for file storage.

4. Blockchain Infrastructure:

The blockchain infrastructure ensures security and transparency in the system by using smart contracts to manage file verification and ownership. Instead of storing entire files, the system stores file hashes, which act as unique digital fingerprints.

2.1.2 FACTORS TO BE CONSIDERED

1. Data Security:

Data security is a critical aspect of the system since it manages sensitive digital files. The system protects data from unauthorized access, tampering, and loss by implementing multiple security measures. Encryption is used to convert files into a secure format before storing them in decentralized storage. The platform also uses secure authentication methods, where users log in with credentials and access is controlled using JWT (JSON Web Tokens). These mechanisms ensure the confidentiality, integrity, and reliability of the system.

2. Data Integrity:

Data integrity ensures that stored files remain accurate, consistent, and unaltered. In this system, integrity is maintained by storing the file hash on the blockchain after the file is uploaded to decentralized storage like IPFS. The generated hash acts as a unique identifier for the file. Since blockchain records are immutable and tamper-proof, the stored hash cannot be changed. If the file is modified, its hash will not match the blockchain record, allowing easy detection of tampering and ensuring file authenticity and reliability.

3. Scalability:

Scalability ensures that the system can handle increasing numbers of users and files without affecting performance. As the platform grows, it must efficiently manage larger workloads and data volumes. To support this, the system uses decentralized storage like IPFS and cloud services, which distribute data across multiple nodes instead of a single server. This improves storage efficiency and system performance. Additionally, a modular backend architecture and blockchain integration allow the system to scale smoothly while maintaining reliability and availability.

4. Access Control:

Access control ensures that only authorized users can access and manage system data**,** protecting it from unauthorized access. In this system, users must log in with verified credentials before performing actions like uploading, viewing, or downloading files. The system uses token-based authentication such as JWT to verify user identity.

5. Storage Optimization:

It is important in blockchain systems because storing large files directly on the blockchain is costly and inefficient. To solve this, the system stores only the file hash or metadata on the blockchain, while the actual files are stored in cloud or decentralized storage such as IPFS. The stored hash acts as a unique identifier to verify file authenticity and integrity. This approach reduces blockchain storage load while maintaining security and verification. As a result, the system improves cost efficiency, performance, and scalability.

6. Performance:

It is essential for ensuring a smooth and efficient user experience in the system. The platform should support fast file uploads and downloads so users can quickly store and retrieve files. It must also handle multiple user requests simultaneously without delays. Maintaining low latency helps reduce the time required for processing requests, communicating with storage services, and verifying data through blockchain.

7. Security and Monitoring:

Security and monitoring ensure that the system operates safely and transparently. All communication between the client and server uses HTTPS, which encrypts data and protects sensitive information during transmission. The system also implements authentication and authorization to verify user identity and control access to resources.

2.1.3 TECHNOLOGICAL APPROACHES

The system uses a combination of modern technologies to build a secure and transparent platform for managing digital files. At the core of the system is blockchain technology, specifically the Ethereum network, which is used to maintain a decentralized and tamper-proof record of file transactions. Smart contracts written in Solidity are deployed on the blockchain to automate important operations such as storing file hashes and recording transaction details. These smart contracts eliminate the need for manual verification and reduce the chances of human error. Because blockchain data is immutable, once a file hash is stored, it cannot be modified or deleted, which ensures the integrity and reliability of the system.

The system follows a hybrid storage approach where the actual files are stored off chain while only their cryptographic hashes are recorded on the blockchain. Storing complete files directly on the blockchain would be inefficient and expensive, so the system uses storage solutions such as IPFS or cloud storage services like AWS to store the files. When a file is uploaded, the storage system generates a unique hash value or content identifier that represents the file. This hash is then sent to the blockchain through the smart contract and permanently recorded. Because every file has a unique hash, even the smallest modification in the file will produce a different hash value, allowing the system to detect any unauthorized changes.

The backend of the system acts as a bridge between the frontend interface, blockchain network, and the storage services. It is responsible for handling requests from users, processing file uploads, communicating with the IPFS or cloud storage network, and interacting with the blockchain through smart contracts. Backend APIs manage tasks such as user authentication, transaction processing, and hash storage. Technologies like Node.js and Express.js are commonly used to build these APIs. manage tasks such as user authentication, transaction processing, and hash storage. Technologies like Node.js and Express.js are commonly used to build these APIs. By separating the backend logic from the frontend interface, the system becomes easier to manage, maintain, and scale.

2.1.4 CHALLENGES AND OPPORTUNITTIES

The implementation of a blockchain-based cloud storage system presents several technical challenges that must be carefully addressed during system development. One of the primary challenges is blockchain scalability**.** As the number of users and transactions increases, blockchain networks may experience difficulty in processing a large volume of transactions quickly. Public blockchain networks such as Ethereum have limitations in transaction throughput, which can lead to network congestion and slower performance. When many users attempt to store or verify file hashes simultaneously, the system may experience delays, making scalability an important factor to consider during system design.

Another significant challenge is the high transaction cost, commonly referred to as gas fees. Every interaction with the blockchain, such as storing a file hash or executing a smart contract function, requires a transaction that consumes gas. When network usage is high, these fees can increase significantly, making frequent transactions expensive. For systems that require regular file uploads and verifications, this cost can become a major limitation. Developers must therefore optimize smart contracts and reduce unnecessary blockchain interactions to keep the operational cost manageable.

The integration of multiple technologies also adds complexity to the system architecture. A blockchain-based file management platform combines several components, including blockchain networks, smart contracts, decentralized or cloud storage systems, backend APIs, and frontend interfaces. Ensuring that all these components communicate effectively requires careful system design and proper testing. Developers must manage issues related to compatibility, data synchronization, and secure communication between different layers of the system.

Another challenge relates to data privacy and security**.** In many public blockchain networks, transaction data is visible to all participants. If sensitive information is stored directly on the blockchain, it could potentially expose confidential data. To avoid this issue, the system stores only cryptographic hashes of files on the blockchain rather than the files themselves.

3. EXISITING SYSTEM

The existing system of Blockchain Integration with Cloud Storage for Secure and Transparent File Management begins with the user accessing the system and completing authentication. Authentication ensures that only authorized users can access the platform, protecting the system from unauthorized access. Once the user successfully logs in, they are allowed to upload files into the system. This initial stage verifies the identity of the user and establishes a secure environment before any file operations are performed.

After the file is uploaded, the system creates a blockchain structure to manage and track the file securely. The uploaded file is divided into smaller blocks, which improves efficiency and security in storage and management.

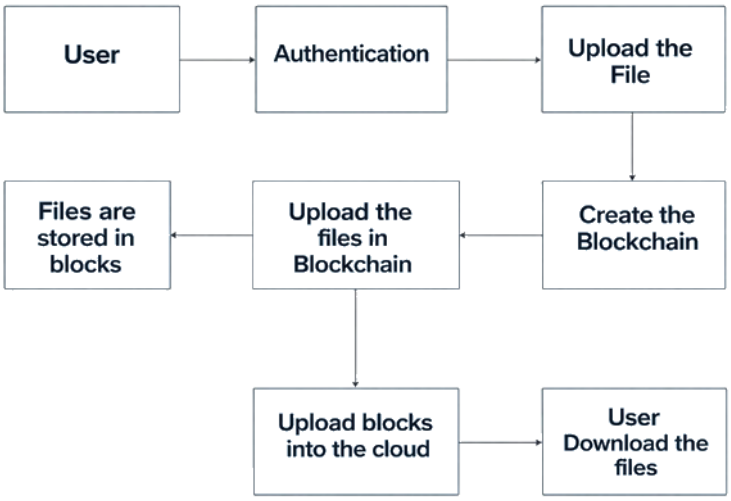


Figure 3.0 – Existing System

These file blocks are then recorded and linked through blockchain technology, ensuring transparency, immutability, and traceability of the data. Because blockchain maintains a distributed ledger, any modification attempt can be detected easily, thereby increasing the reliability and integrity of the stored files.

3.1 DISADVANTAGES OF EXISITING SYSTEM

One major limitation is the high computational and storage requirements**.** Blockchain technology requires significant processing power to create and maintain blocks, especially when handling large amounts of data. As the number of files and transactions increases, the size of the blockchain ledger also grows, which can lead to increased storage costs and slower system performance.

Another disadvantage is increased system complexity and implementation cost. Integrating blockchain with cloud storage requires advanced infrastructure, specialized software, and skilled technical knowledge. Developing and maintaining such a system can be expensive for organizations, especially small or medium-scale institutions. The complexity of blockchain networks can also make system management and troubleshooting more difficult.

The system may also experience performance and scalability issues. Since blockchain transactions require validation and consensus mechanisms, processing file-related operations can take more time compared to traditional cloud storage systems. This may lead to slower upload and download speeds, especially when many users are accessing the system simultaneously. As a result, the system may face challenges in handling large-scale applications efficiently.

Another challenge is network latency and energy consumption. Blockchain systems rely on consensus mechanisms where multiple nodes in the network verify and validate each transaction before it is added to the blockchain. This verification process can introduce delays, especially when the network is handling a large number of transactions simultaneously. In addition, some blockchain mechanisms require significant computational power, which increases energy consumption and operational costs. . Compared to traditional centralized cloud systems, blockchain networks may therefore consume more resources to maintain security and reliability.

4. PROPOSED SYSTEM

The diagram shows the architecture of a Blockchain-based Secure Cloud File Management System. The process starts with the user client, where the user uploads a file through the File Upload Interface. During this stage, key management and user authentication are performed to ensure that only authorized users can upload files. After authentication, the file enters the cloud storage network, where it undergoes file chunking. This means the file is divided into smaller parts (chunks) using a data.

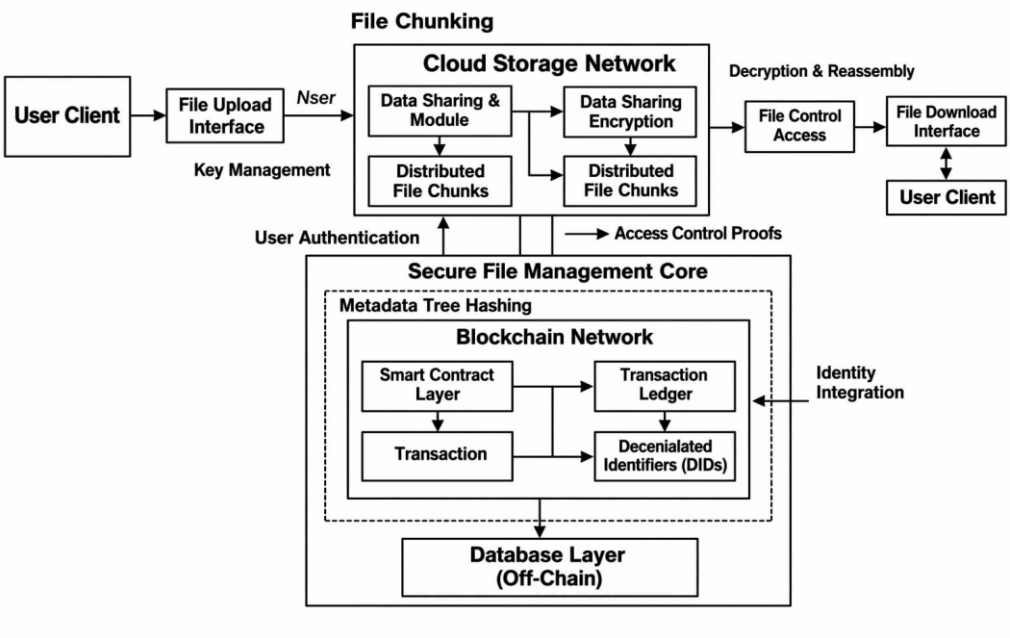


Figure 4.0 – Proposed System.

The Secure File Management Core connects the cloud storage system with the blockchain network. The blockchain stores important information such as file metadata, access permissions, and transaction records using smart contracts. Each file operation (upload, access, or modification) is recorded in the transaction ledger, which ensures transparency and prevents data tampering. The system may also use Decentralized Identifiers (DIDs) to manage digital identities and verify users. Some additional data and metadata can be stored in an off-chain database layer to improve system efficiency and reduce blockchain storage costs. When a user wants to retrieve a file, the request goes through file control and access verification.

4.1 ADVANTAGES OF PROPOSED SYSTEM

The proposed system introduces several improvements that enhance the security, efficiency, and reliability of digital file management. One of the major advantages of the system is strong data security achieved through the combination of encryption techniques and blockchain technology. Before storing files in the cloud storage network, the files are divided into smaller chunks and encrypted to protect their contents. This encryption ensures that even if unauthorized users gain access to the stored data, they cannot read or interpret the information without the correct decryption keys. At the same time, blockchain technology records metadata and transaction details through smart contracts and a distributed ledger. These records create a permanent and tamper-proof history of file operations, ensuring transparency and preventing unauthorized modification of stored data.

Another important advantage of the proposed system is improved access control and identity management. The system integrates user authentication mechanisms with decentralized identifiers (DIDs) and blockchain-based smart contracts. This allows the platform to verify the identity of users before allowing them to interact with the storage system. Only authorized users are permitted to upload, access, or download files, which strengthens the overall security of the platform. In addition, the system uses access control proofs and secure key management mechanisms to manage permissions and protect sensitive data.

The proposed system also improves scalability and reliability by adopting a hybrid architecture that separates blockchain operations from data storage. Instead of storing entire files on the blockchain, the system stores only metadata and transaction records on the blockchain, while the actual file data is stored off chain in cloud or decentralized storage networks. Files are divided into multiple distributed chunks and stored across different storage nodes, which improves storage efficiency and reduces the risk of data loss. Even if one storage node fails, the remaining nodes can still provide access to the stored data.

4.2 ALGORITHM IMPLEMENTATION

In the Blockchain Integration with Cloud Storage for Secure and Transparent File Management project, several algorithms and techniques are used to ensure security, integrity, and efficient file storage.

1. AES (Advanced Encryption Standard) Algorithm:

AES encryption is used to secure the files before storing them in cloud storage. When a user uploads a file, the system encrypts the data using AES so that unauthorized users cannot read the file. During download, the file is decrypted using the same key, ensuring that only authorized users can access the original content.

2. SHA-256 Hashing Algorithm:

The SHA-256 (Secure Hash Algorithm) is used to generate a unique hash value for each file. This hash acts as a digital fingerprint of the file and is stored on the blockchain. If the file is modified or tampered with, the hash value will change, allowing the system to detect any unauthorized changes.

3. File Chunking / Data Sharding Algorithm:

The system divides large files into smaller chunks or blocks before storing them in cloud storage. This method improves data distribution, reliability, and storage efficiency. If needed, these chunks are later combined to reconstruct the original file during download.

4. Authentication Algorithm (Token based Authentication):

The project uses token-based authentication (such as JWT) to verify user identity. After login, a secure token is generated, which allows the user to access the system without repeatedly logging in while maintaining security.

5. SYSTEM ARCHITECTURE

5.1 SYSTEM ARCHITECTURE

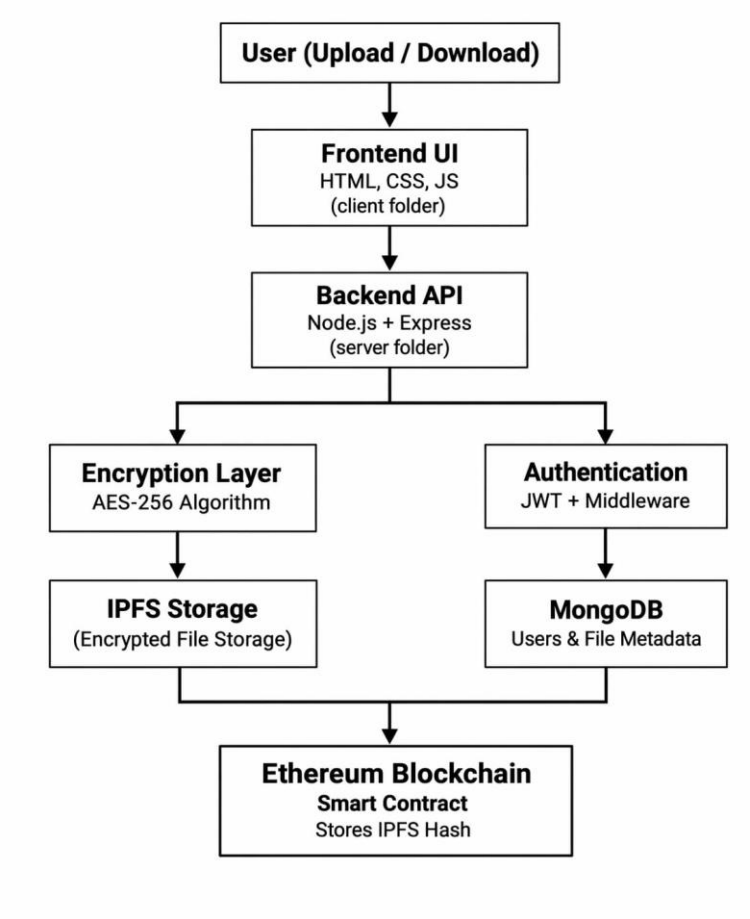


Figure 5.1 – System Architecture.

The complete architecture of the system designed for secure file storage using blockchain and decentralized cloud storage. This architecture integrates modern web technologies, encryption techniques, distributed storage, and blockchain networks to ensure that files are stored securely and remain tamper-proof. The system is designed in a layered structure so that each component performs a specific function such as user interaction, data processing, encryption, authentication, and storage. By separating these responsibilities, the system becomes more secure, scalable, and easier to maintain.

The process begins when a user interacts with the system through the frontend interface. The frontend is developed using HTML, CSS, and JavaScript, which provides an interactive web interface where users can register, log in, upload files, and download previously stored.

One of the most important components in the backend is the Encryption Layer, which uses the AES-256 encryption algorithm. AES-256 is a highly secure symmetric encryption standard widely used in modern security systems. Before a file is stored, the backend encrypts the file so that the original data becomes unreadable without the correct decryption key. This step ensures data confidentiality, meaning that even if someone accesses the stored file, they cannot understand its contents. Once the file is encrypted, it is uploaded to IPFS (interplanetary File System). IPFS is a decentralized peer-to-peer storage system that stores files across multiple nodes rather than in a single centralized server. Each file stored in IPFS is assigned a unique content-based hash, which acts as a permanent identifier for retrieving that file.

At the same time, the system uses an Authentication and Authorization mechanism to ensure that only valid users can access the system. Authentication is implemented using JWT (JSON Web Token) along with middleware in the backend. When a user logs in, the server generates a JWT token that is sent to the client. This token is included in future requests to verify the user's identity. The middleware checks the token before allowing access to protected resources such as file upload or download operations. Additionally, MongoDB is used as the database to store user information, login credentials, and file metadata such as file names, upload timestamps, and associated IPFS hashes. This database helps manage system data efficiently while supporting quick retrieval of information.

Another important part of the architecture is the Ethereum Blockchain integration. After the encrypted file is stored in IPFS and the hash is generated, the backend sends this hash to a smart contract deployed on the Ethereum network. The smart contract records the IPFS hash permanently on the blockchain ledger. Since blockchain technology is immutable, the stored hash cannot be modified or deleted once it is recorded. This feature ensures data integrity, meaning users can verify that the file stored in IPFS has not been altered. If someone attempts to change the file, the generated hash will be different, making it easy to detect tampering. During the file retrieval process, the user sends a request to download a specific file through the frontend interface.

5.2 DATA FLOW DIAGRAMS

5.2.1 LEVEL – 0 CONTEXT DIAGRAM

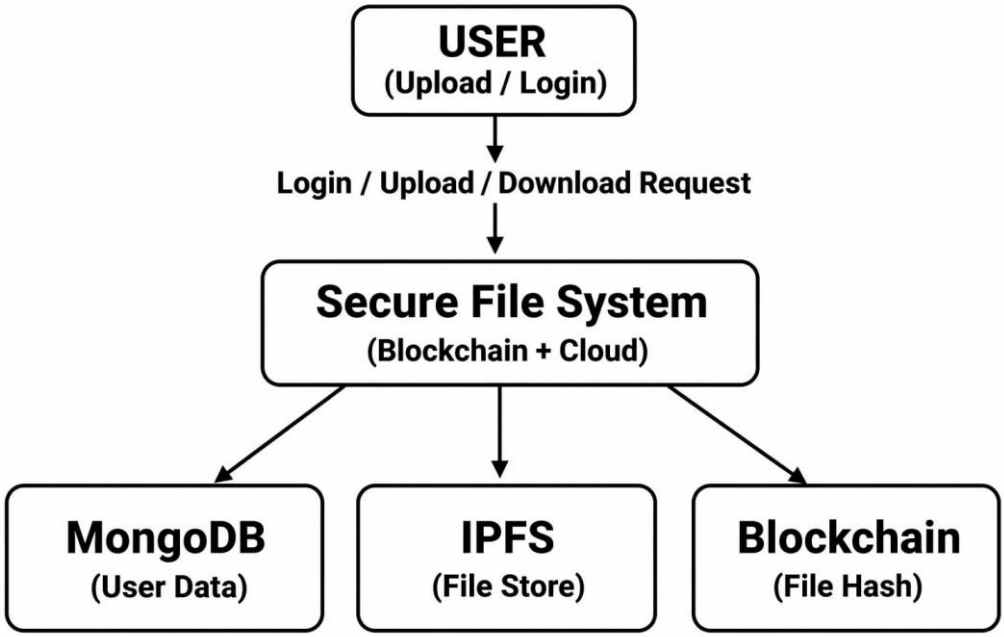


Figure 5.2.1 – Level – 0 Context Diagram.

The diagram represents the high-level workflow of a Secure File System that integrates Blockchain technology with Cloud Storage. It illustrates how users interact with the system and how different technologies such as MongoDB, IPFS, and Blockchain work together to securely store and manage files. The architecture focuses on improving data security, transparency, and decentralized storage, ensuring that files remain protected from unauthorized access or tampering. At the top of the diagram is the **User**, who interacts with the system by performing operations such as login, uploading files, and downloading files. The user sends requests to the system through a secure interface.

The Secure File System (Blockchain + Cloud) serves as the core layer that integrates cloud storage with blockchain technology. This system processes the user's requests and determines how the data should be stored or retrieved. When a user uploads a file, the system first performs security checks and encryption processes before storing the file.

The system also manages communication with external services such as databases, decentralized storage networks, and blockchain networks. By combining these technologies, the secure file system ensures that the stored data remains confidential, tamper-proof, and easily verifiable.

The system connects to MongoDB, which functions as the database for storing user-related information and metadata. This includes details such as user accounts, login credentials, file names, file upload timestamps, and references to stored files. MongoDB is a NoSQL database that provides flexible and scalable data storage, making it suitable for handling large amounts of application data. It ensures that user information and file details can be quickly accessed and managed by the system.

The second component connected to the secure file system is IPFS (interplanetary File System), which is responsible for file storage. IPFS is a decentralized storage network that distributes files across multiple nodes instead of storing them on a single server. When a file is uploaded, it is stored in IPFS and assigned a unique content-based hash that identifies the file. This hash acts as the file's address within the IPFS network and is used later when the file needs to be retrieved.

The third component is the Blockchain, which is used to store the file hash generated by IPFS. Instead of storing the entire file on the blockchain (which would be inefficient and expensive), only the hash of the file is stored in a blockchain transaction. This hash is stored through a smart contract, ensuring that the file record is permanent and cannot be modified. Since blockchain technology is immutable, the stored hash acts as proof of the file's authenticity and integrity.

5.2.2 LEVEL – 1 SYSTEM PROCESSES

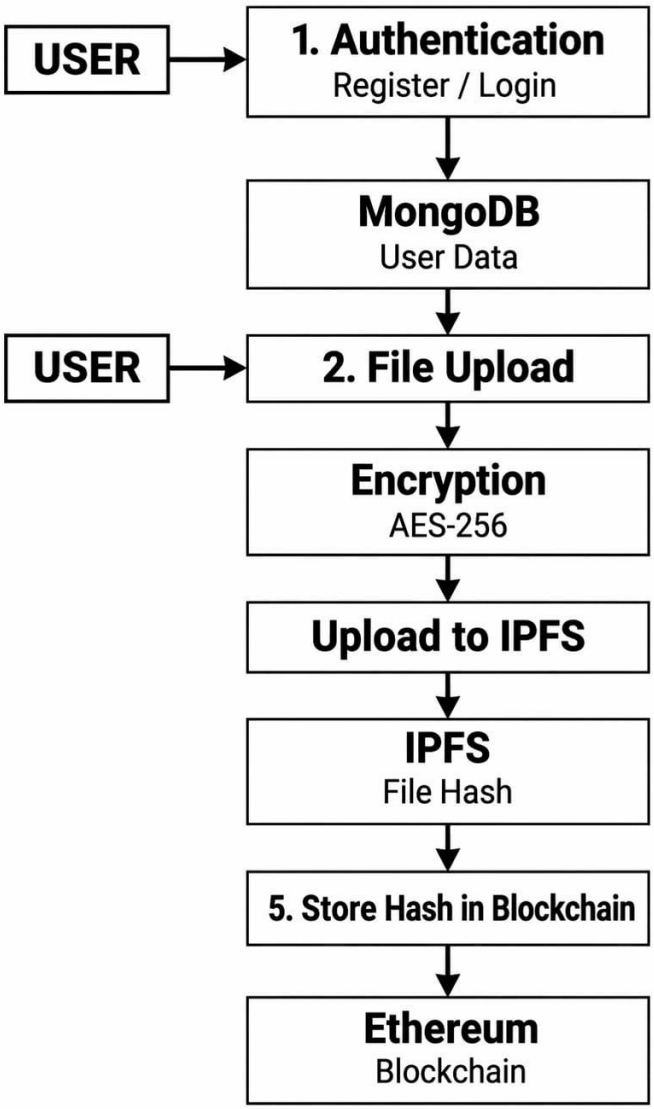


Figure 5.2.2 – Level – 1 System Processes.

The step-by-step workflow of the secure file storage system used in the project "Blockchain Integration with Cloud Storage for Secure and Transparent File Management." It explains how a file moves through different stages of the system, starting from user authentication and ending with secure storage of the file hash on the Ethereum blockchain. This process ensures that files are stored securely, encrypted properly, and verified using blockchain technology to maintain integrity and transparency.

After successful authentication, the user proceeds to the File Upload stage. In this step, the user selects a file from their device and uploads it to the system through the frontend interface. Once the file reaches the backend server, it is processed before being stored. The system ensures that the uploaded file is handled securely and prepares it for encryption and decentralized storage.

The next step is the Encryption Process, where the uploaded file is encrypted using the AES-256 encryption algorithm. AES-256 is one of the most secure symmetric encryption standards used in modern security systems. This encryption converts the original file into an unreadable encrypted format, ensuring that even if unauthorized users gain access to the stored file, they will not be able to understand its content without the correct decryption key. Encryption therefore protects the confidentiality and privacy of user data.

After encryption, the system performs Upload to IPFS (Interplanetary File System). IPFS is a decentralized storage network that stores files across multiple nodes instead of a single centralized server. When the encrypted file is uploaded to IPFS, the system generates a unique IPFS hash, also known as a Content Identifier (CID). This hash represents the file's unique identity in the IPFS network. Instead of locating files by server location, IPFS retrieves files using this hash, which ensures efficient and distributed storage.

Once the IPFS hash is generated, the system proceeds to the next step, which is storing the hash in the blockchain. Overall, this workflow demonstrates how the system integrates authentication, encryption, decentralized storage (IPFS), and blockchain verification to create a highly secure file management solution.

This process ensures that files are stored securely, encrypted properly, and verified using blockchain technology to maintain integrity and transparency. When the encrypted file is uploaded to IPFS, the system generates a unique IPFS hash, also known as a Content Identifier (CID). This hash represents the file's unique identity in the IPFS network. Instead of locating files by server location, IPFS retrieves files using this hash, which ensures efficient and distributed storage.

5.2.3 LEVEL – 2 FILE UPLOAD DETAILED FLOW

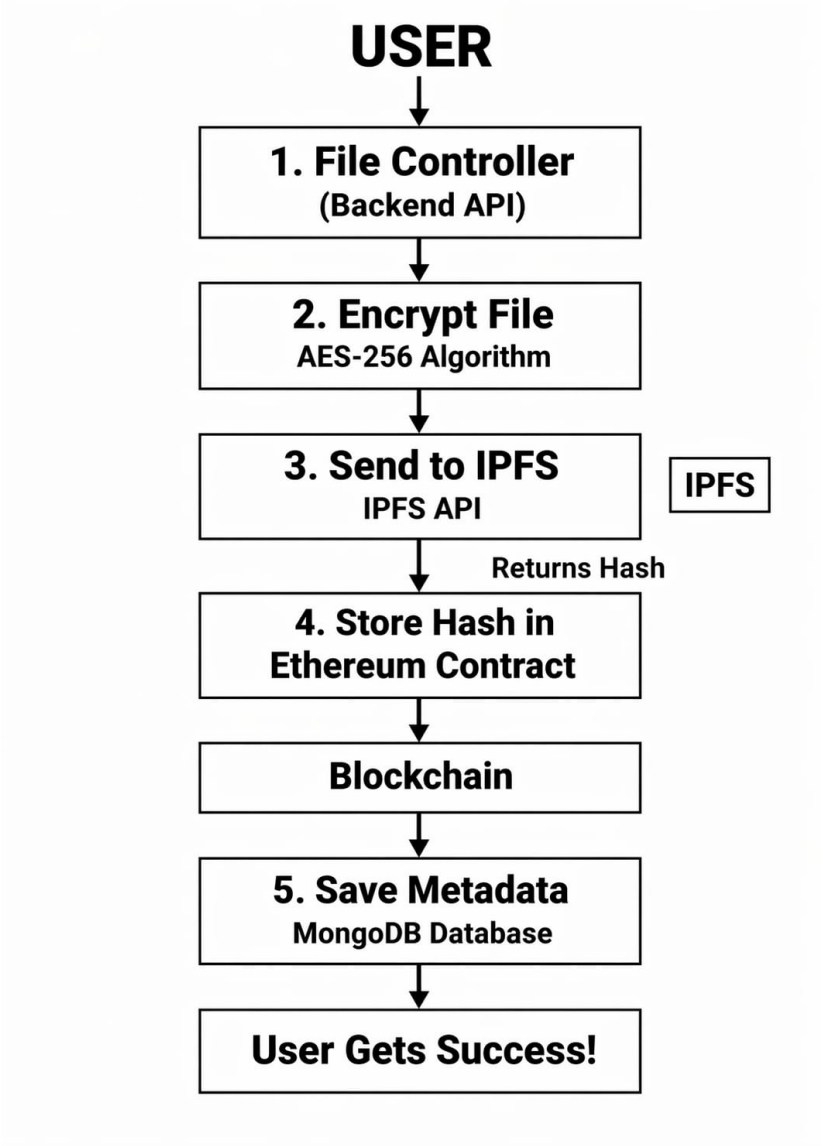


Figure 5.2.3- Level – 2 File Upload Detailed Flow.

the detailed workflow of the file upload process in the Blockchain-based Secure File Storage System. It shows how a user's file moves through different stages such as backend processing, encryption, decentralized storage using IPFS, blockchain verification, and metadata storage. Each step in this workflow ensures that the file remains secure, verifiable, and tamper-proof throughout the storage process.

The process begins with the User, who initiates the file upload request through the system interface. The user selects a file and sends it to the server. This request is first handled by the File Controller (Backend API), which acts as the central processing unit of the application.

The file controller, usually implemented using technologies such as Node.js and Express, receives the uploaded file, validates it, and manages the workflow required to store the file securely. It coordinates the interaction between encryption modules, storage services, blockchain components, and the database.

Once the backend receives the file, the system proceeds to the file encryption stage. In this step, the file is encrypted using the AES-256 encryption algorithm, which is a highly secure symmetric encryption standard. The encryption process converts the original file into an unreadable format so that unauthorized users cannot access the content. Even if someone gains access to the stored data, they will only see encrypted information that cannot be interpreted without the proper decryption key. This step ensures the confidentiality and security of sensitive data.

After encryption, the encrypted file is sent to IPFS (interplanetary File System) using the IPFS API. IPFS is a decentralized peer-to-peer file storage network that stores files across multiple distributed nodes rather than on a single centralized server. When the file is uploaded to IPFS, the network generates a unique content identifier (hash) for that file. This hash acts as the permanent address of the file within the IPFS network. Because the hash is based on the file's content, any modification to the file will generate a completely different hash.

Once the IPFS network returns the file hash, the system moves to the blockchain integration step. In this stage, the hash generated by IPFS is stored in a smart contract deployed on the Ethereum blockchain.

6. SYSTEM SPECIFICATIONS

6.1 HARDWARE SPECIFICATIONS

The Hardware Specifications table outlines the minimum and recommended hardware requirements needed to run the Blockchain Integration with Cloud Storage for Secure and Transparent File Management System efficiently. These components ensure smooth system performance during development, testing, and deployment.

The system requires a processor such as Intel Core i3 or AMD Ryzen 5 or higher to handle tasks like encryption, backend processing, blockchain interaction, and database operations. A minimum of 8GB RAM (16GB recommended) is needed to run multiple services such as the backend server, MongoDB database, and development tools simultaneously without performance issues.

For storage, the system requires 256GB or 512GB SSD/HDD, with SSD recommended for faster performance and quicker data access. A stable internet connection is also necessary because the system interacts with cloud storage, IPFS networks, and blockchain services.

S.NO	COMPONENTS	SPECIFICATIONS
1.	Processor	Intel Core i3 or above / Ryzen 5
2.	RAM	8GB (Recommended 16GB)
3.	Storage	256/514 SSD/HDD
4.	Network	Internet Connection
5.	Input Devices	Keyboard, Mouse
6.	Display	Standard Monitor

Table 6.1 – Hardware Specifications.

6.2 SOFTWARE SPECIFICATIONS

S.NO	SOFTWARE / TOOL	DESCRIPTION
1.	Windows 10/11	Operating System
2.	Node.js	Backend runtime
3.	Express.js	Backend framework
4.	MongoDB	Database
5.	Ethereum	Blockchain platform
6.	Solidity	Smart contract language
7.	Hardhat	Blockchain development environment
8.	Web3.js	Blockchain communication
9.	IPFS	Distributed file storage
10.	JWT	Authentication
11.	AES-256	File encryption
12.	VS Code	Development IDE
13.	Chrome / Firefox	Web browser

Table 6.2 – Software Specifications.

The listed software components support different functions such as application development, backend processing, database management, blockchain integration, encryption, and user interaction in the system. The system runs on Windows 10 or Windows 11, which provides the environment for installing and running development tools. The backend is developed using Node.js as the runtime environment and Express.js as the framework for building APIs and handling server-side operations. MongoDB is used as the database to store user information and file metadata. For blockchain functionality, the system uses the Ethereum platform, where Solidity is used to write smart contracts and Hardhat is used for developing and testing these contracts. Communication between the application and blockchain is handled using Web3.js.

6.3 LIBRARIES USED

The Libraries Used table lists the various software libraries required to build and support the system's functionality. These libraries help in tasks such as backend API development, database connectivity, user authentication, password encryption, file uploads, blockchain interaction, and decentralized storage integration.

S.NO	LIBRARY	PURPOSE
1.	Express	Backend web framework used to build the API server.
2.	Mongoose	Connects the Node.js server with MongoDB database.
3.	Jsonwebtoken (JWT)	Handles user authentication and authorization.
4.	Bcrypt	Encrypts and hashes user passwords.
5.	Bcryptjs	Alternative password hashing library
6.	Cors	Enables communication between frontend and backend
7.	Dotenv	Loads environment variables from .env files
8.	Multer	Handles file uploads from users
9.	IPFS-http-client	Connects the system to IPFS decentralized storage
10.	Ethers	Interacts with Ethereum blockchain and smart contracts
11.	Morgan	Logs server requests for debugging
12.	Nodemailer	Sends emails (e.g., password reset or notifications)
13.	Qrcode	Generates QR codes for file verification or sharing
14.	Speakeasy	Implements Two-Factor Authentication (2FA)
15.	Winston	Advanced logging for server events and errors
16.	Axios	Makes HTTP requests between services.

Table 6.3 – Libraries Used.

7. EXPERIMENTAL DESIGN DIAGRAMS

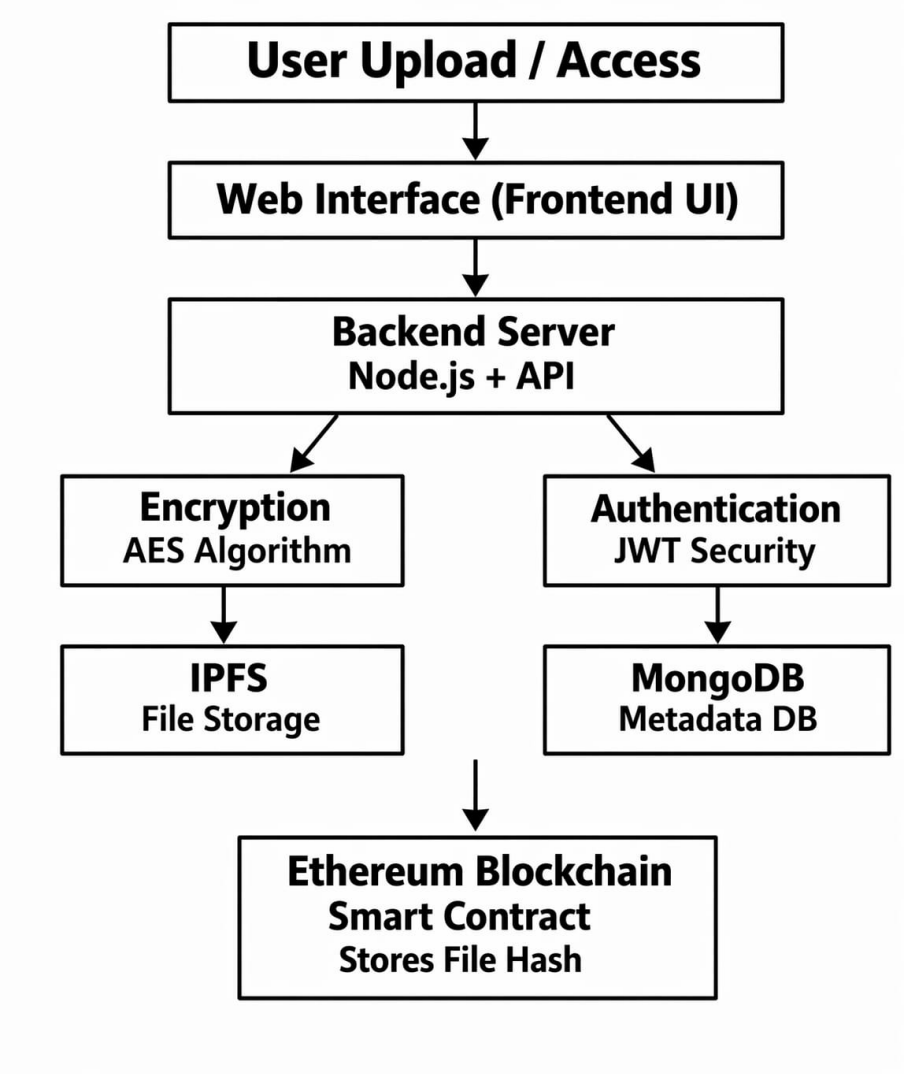


Figure 7.0 – Experimental Design Diagram.

The experimental design of the Machine Learning–based Intrusion Detection System (IDS) is organized as a structured and systematic pipeline aimed at developing a highly accurate, scalable, and adaptive cybersecurity solution. The process begins with dataset acquisition, where large and diverse datasets such as NSL-KDD, KDD Cup 99, or real-time network traffic data are collected. These datasets contain records of both normal network activities and different types of cyberattacks. After collecting the datasets, the system performs a data preprocessing phase to prepare the data for machine learning analysis. This stage involves several important steps such as data cleaning, removal of noise, handling missing or inconsistent values, normalization of numerical data, and encoding of categorical features.

These preprocessing steps improve the quality of the dataset and ensure that the information is structured in a format suitable for machine learning algorithms. Proper preprocessing helps eliminate errors and inconsistencies that could negatively affect the performance of the intrusion detection model.

Once the dataset is cleaned and prepared, the next step is feature engineering, which involves selecting and extracting the most relevant attributes that contribute to detecting network intrusions. Network datasets often contain a large number of features, some of which may be redundant or irrelevant. Techniques such as correlation analysis, Principal Component Analysis (PCA), and information gain are used to identify important features and reduce the dimensionality of the dataset. This process improves computational efficiency, reduces training time, and enhances the model's ability to learn meaningful patterns from the data without sacrificing detection accuracy.

After feature optimization, the dataset is divided into training and testing subsets using standard splitting techniques such as 70:30 or 80:20 ratios. The training dataset is used to train the machine learning models, while the testing dataset is used to evaluate the performance of the trained models. This separation helps ensure that the models are validated using unseen data and reduces the risk of overfitting, where a model performs well on training data but poorly on new data.

The model development stage uses a hybrid machine learning approach that combines both supervised and unsupervised learning techniques. Supervised learning algorithms such as Decision Trees, Random Forest, and Support Vector Machines (SVM) are trained on labelled datasets to accurately identify known attack patterns and classify network traffic as normal or malicious. At the same time, unsupervised learning techniques, including K-Means clustering and anomaly detection methods, are used to identify unusual network behaviors that may represent unknown or zero-day attacks.

7.1 USE CASE DIAGRAM

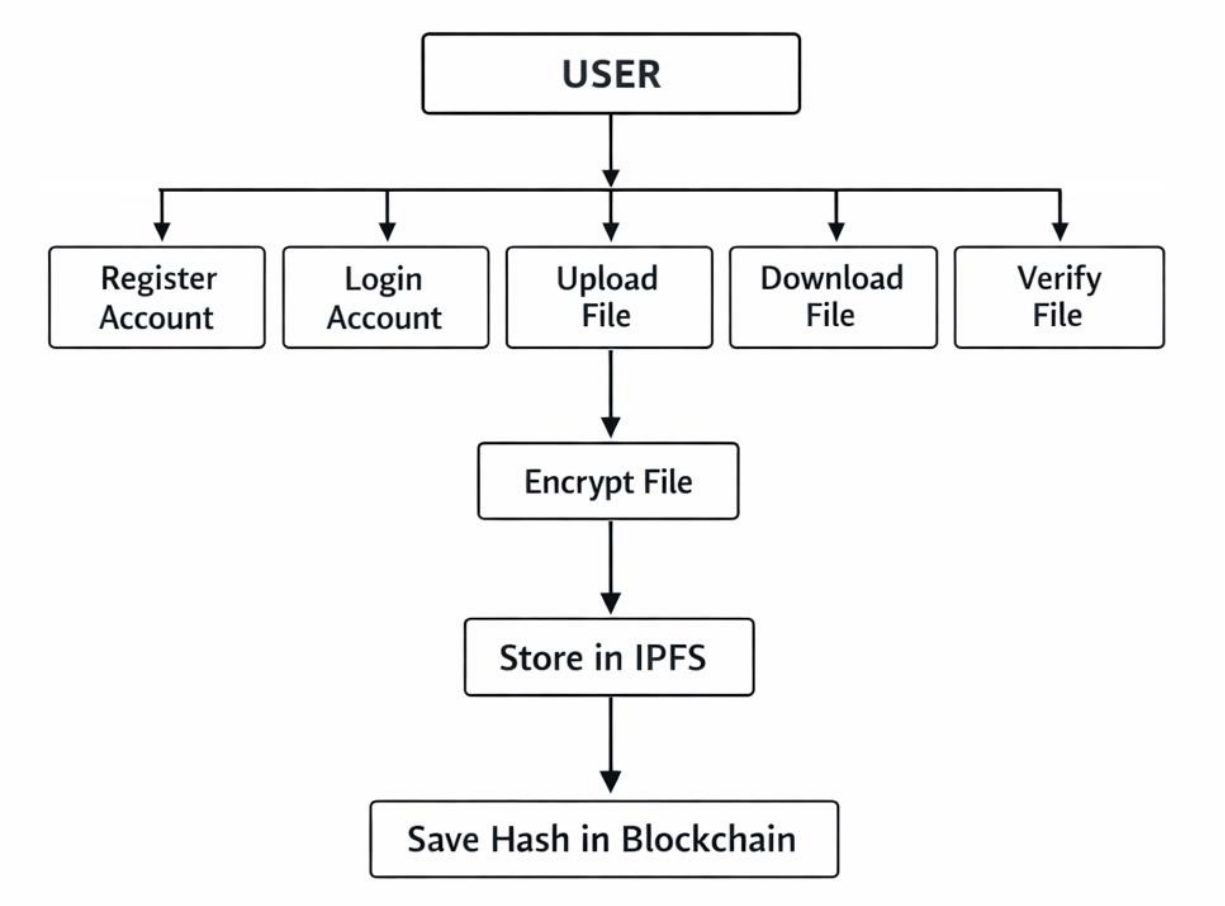


Figure 7.1 – Use Case Diagram.

The diagram represents the complete functional workflow of the Blockchain-Based Secure File Storage System from the user's perspective. It demonstrates how users interact with the system and how the system securely manages file storage using encryption, decentralized cloud storage (IPFS), and blockchain verification. The main goal of this system is to ensure that files are stored securely while maintaining data confidentiality, integrity, transparency, and reliability. By combining multiple technologies, the system provides a secure environment for storing and retrieving digital files.

After registration, users must log in to the system to access its services. The login process verifies the user's credentials against the stored database records. Once authenticated, the system may generate a secure session or token (such as JWT) that allows the user to perform actions within the system.

This authentication mechanism prevents unauthorized users from accessing the platform and ensures that only valid users can upload or download files.When a user chooses to upload a file, the system begins a multi-step security process. The uploaded file is first sent to the backend server, where it undergoes encryption using the AES-256 algorithm**.** AES-256 is a widely used encryption standard that converts readable data into an encrypted format using a secret key. This ensures that even if someone gains unauthorized access to the stored file, they will not be able to read its contents without the decryption key. Encryption plays a critical role in protecting sensitive information and maintaining user privacy.

After encryption, the encrypted file is uploaded to IPFS (interplanetary File System)**.** IPFS is a decentralized file storage protocol that distributes files across multiple nodes in a peer-to-peer network. Unlike traditional cloud storage systems that rely on centralized servers, IPFS stores files in a distributed manner. Each stored file is assigned a unique cryptographic hash, also known as a Content Identifier (CID)**.** This hash acts as a permanent reference to the file and is used to retrieve it later. Since the hash is generated from the file's content, any modification to the file will automatically produce a different hash value.

Once the file is successfully stored in IPFS, the generated hash is recorded in the blockchain network through a smart contract. In most implementations, platforms such as the Ethereum blockchain are used for this purpose. Instead of storing the entire file on the blockchain, which would be costly and inefficient, only the IPFS hash is stored. This approach significantly reduces storage costs while still ensuring that the file's authenticity can be verified. The blockchain provides an immutable and transparent ledger, meaning that once the hash is recorded, it cannot be altered or removed.

7.2 CLASS DIAGRAM

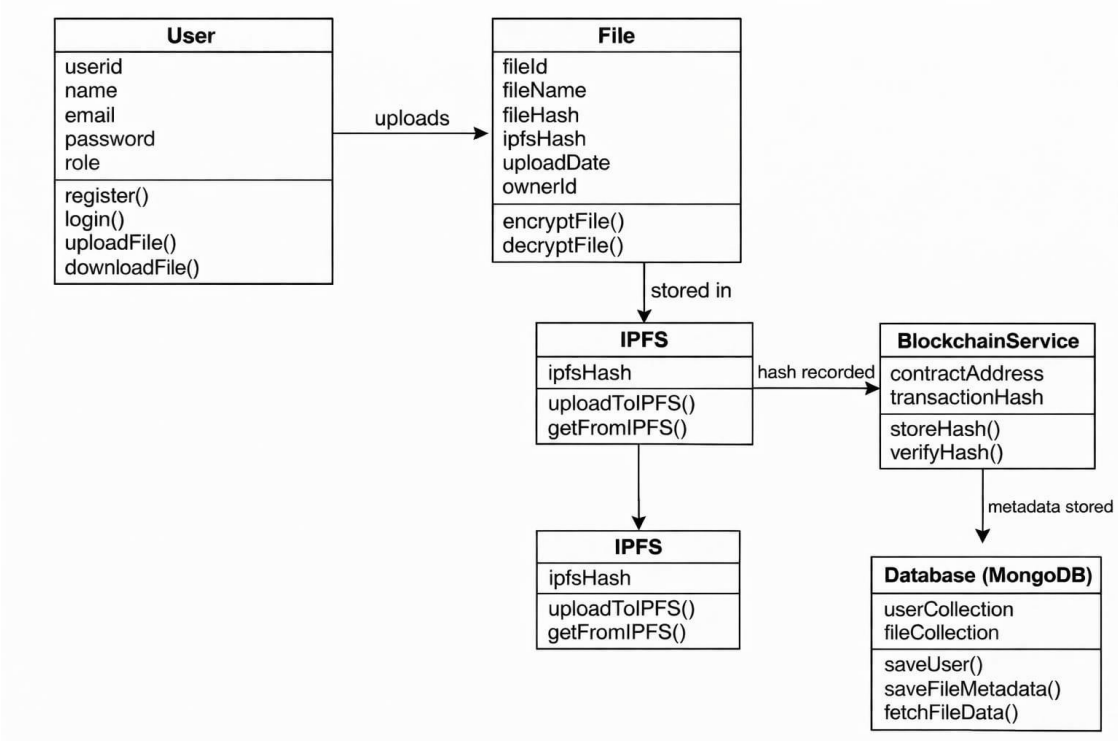


Figure 7.2 – Class Diagram.

The diagram represents a class diagram of the Blockchain-Based Secure File Storage System, showing the main components of the system and how they interact with each other. It describes the structure of the system by defining different classes such as User, File, IPFS, Blockchain Service, and Database (MongoDB) along with their attributes and functions. This design helps in understanding how files are uploaded, stored securely, and verified using blockchain technology.

The first class in the diagram is the User class, which represents the individuals who interact with the system. The User class contains attributes such as userId, name, email, password, and role. These attributes store information about each registered user in the system. The class also contains functions such as register(), login(), uploadFile(), and downloadFile()**.** The register**()** function allows a new user to create an account, while login() verifies the user's credentials to allow access to the system. The uploadFile() function allows the user to upload files to the system, and downloadFile() allows them to retrieve stored files. In this architecture, the user initiates most of the system operations.

The File class represents the files uploaded by users into the system. It includes attributes such as fileId, fileName, fileHash, ipfsHash, uploadDate, and ownerId**.** These attributes store information about each uploaded file, including its identity, storage hash, upload time, and the user who uploaded it. The File class also includes functions such as encryptFile() and decryptFile()**.** The encryptFile() function is used to encrypt the file using algorithms such as AES before storing it in the decentralized storage system. The decryptFile() function allows the encrypted file to be converted back into its original readable format when a user downloads it.

The next component shown in the diagram is IPFS (InterPlanetary File System)**,** which is responsible for decentralized file storage. The IPFS class contains the attribute ipfsHash**,** which represents the unique identifier generated for each file stored in the IPFS network. It also includes functions such as uploadToIPFS() and getFromIPFS(). The uploadToIPFS() function uploads encrypted files to the IPFS network, and the system generates a unique hash that can be used later to retrieve the file. The getFromIPFS() function retrieves the stored file using its IPFS hash.

Another important component in the diagram is the Blockchain Service class, which manages the interaction with the blockchain network. This class contains attributes such as contractAddress and transactionHash, which represent the blockchain smart contract address and the transaction reference where the file hash is recorded. The class provides functions such as storeHash() and verifyHash()**.** The storeHash() function records the IPFS file hash in the blockchain through a smart contract, ensuring that the record is permanent and tamper-proof. The verifyHash() function allows the system to verify the authenticity of a file by comparing its current hash with the one stored on the blockchain.

The final component in the diagram is the Database class using MongoDB, which stores application data and metadata. The database contains collections such as userCollection and fileCollection**,** which store user details and file metadata respectively. The functions associated with this class include saveUser(), saveFileMetadata(), and fetchFileData()**.**

7.3 SEQUENCE DIAGRAM

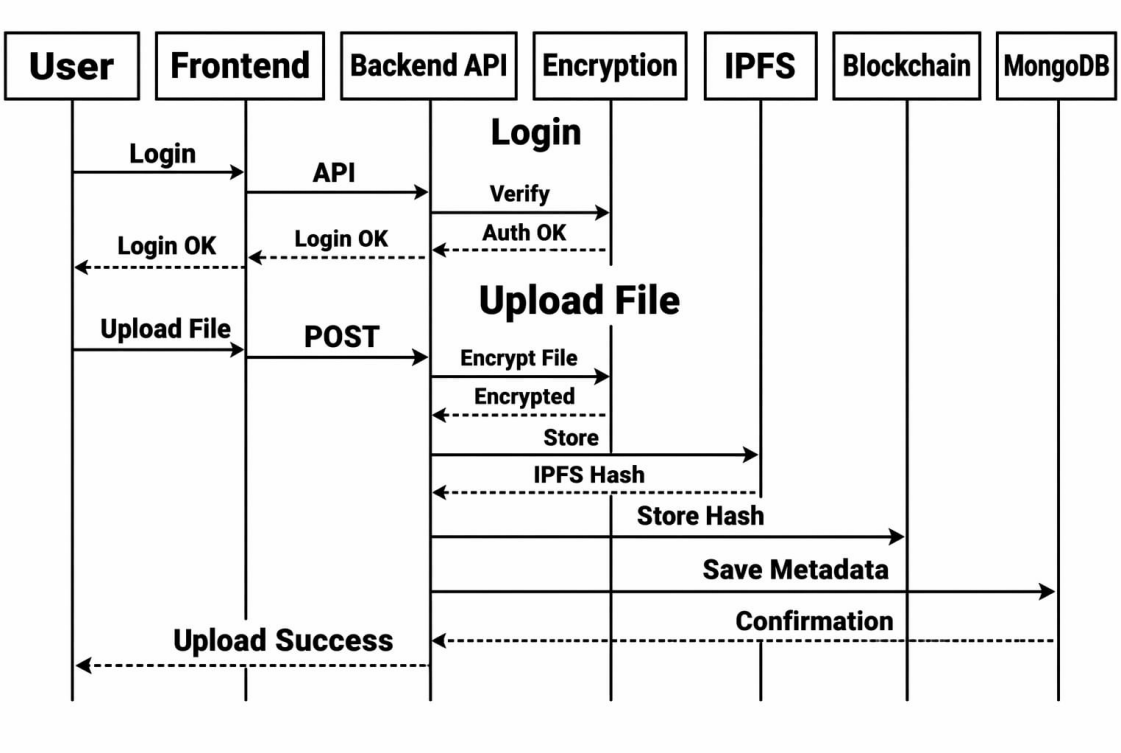


Figure 7.3 – Sequence Diagram.

A sequence diagram illustrates how different components of a system communicate with each other in a specific order to complete a task. In this system, the sequence diagram explains the interaction between the User, Frontend Interface, Backend API, Encryption Module, IPFS Storage, Blockchain Network, and MongoDB Database. It mainly represents two important processes of the system: **User** Login and File Upload. The diagram helps to clearly understand how requests move from one component to another and how each component contributes to secure file management.

At the beginning of the process, the User interacts with the Frontend Interface through the web application. The user enters login credentials such as a username and password. The frontend sends these credentials to the Backend API using an API request. The backend then verifies the user's authentication by communicating with the authentication or encryption module, which checks whether the provided credentials are correct. If the credentials are valid, the backend returns a Login OK response to the frontend. The frontend then notifies the user that the login has been successful.

This authentication process ensures that only authorized users are allowed to access the system.After successfully logging in, the user can perform operations such as uploading a file. The user selects a file from the device and sends the upload request through the frontend interface. The frontend sends a POST request containing the file data to the backend API. Once the backend receives the file, it forwards the file to the Encryption Module, where encryption is applied. Typically, an encryption algorithm such as AES-256 is used to convert the original file into an encrypted format. This step protects the file content from unauthorized access and ensures data confidentiality before it is stored.

After encryption, the backend sends the encrypted file to IPFS (Interplanetary File System) for storage. IPFS stores the file in a decentralized network rather than a centralized server. Once the file is stored, IPFS generates a unique IPFS hash, also known as a content identifier (CID). This hash uniquely represents the stored file and is used as a reference for retrieving the file later. The use of IPFS improves reliability and reduces dependency on centralized storage systems.

Once the IPFS hash is generated, the backend communicates with the Blockchain Network to store this hash in a smart contract. A blockchain transaction is created and recorded on the blockchain ledger. Because blockchain technology provides immutability, the stored hash cannot be modified or deleted once it is recorded. This feature allows users to verify the integrity of the stored file at any time by comparing the current file hash with the hash stored on the blockchain.

After recording the hash on the blockchain, the backend stores file metadata in the MongoDB database. This metadata may include information such as the file name, IPFS hash, upload date, user ID, and blockchain transaction details. MongoDB acts as the system’s database for managing user accounts and file information. Storing metadata in a database allows the system to quickly retrieve file information without repeatedly querying the blockchain network.

7.4 ACTIVITY DIAGRAM

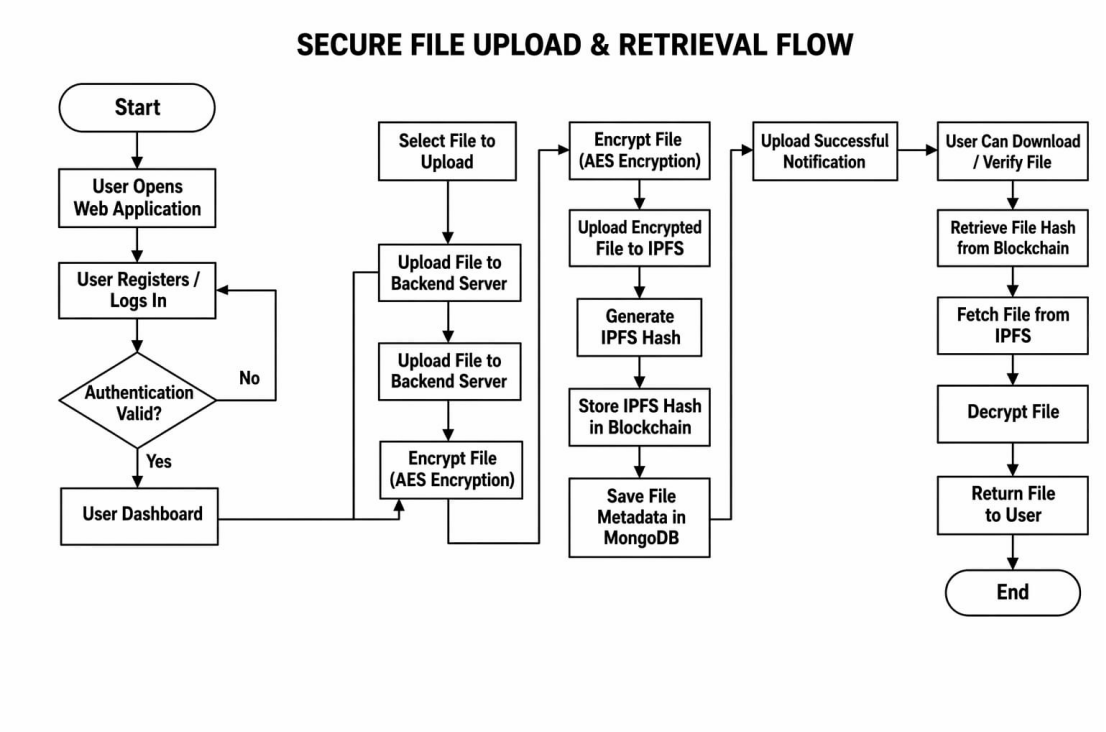


Figure 7.4 – Activity Diagram.

The Secure File Upload and Retrieval Flow describe how a blockchain-based cloud storage system securely stores and retrieves files using multiple technologies such as encryption, decentralized storage (IPFS), blockchain verification, and database management. This workflow ensures that files remain confidential, tamper-proof, and accessible only to authorized users. By combining these technologies, the system provides a secure environment for managing digital files while maintaining transparency and data integrity.

The process begins when the user opens the web application through a browser. At this stage, the user must either register a new account or log in to an existing account by entering credentials such as a username and password. The system then performs an authentication check to verify whether the provided credentials are valid. If the authentication fails, the user is prompted to enter the credentials again. If the authentication is successful, the user gains access to the user dashboard, which provides options to upload files, download files, and verify stored documents

Once the user accesses the dashboard, the user selects a file that needs to be uploaded. The selected file is sent from the frontend interface to the backend server, which handles the processing operations. After receiving the file, the backend performs encryption using the AES (Advanced Encryption Standard) algorithm. Encryption converts the original file into a protected format, ensuring that the file contents cannot be read by unauthorized users even if the file is accessed without permission. This step plays a critical role in maintaining the confidentiality of sensitive data.

After the file is encrypted, the system uploads the encrypted file to IPFS (InterPlanetary File System). IPFS is a decentralized peer-to-peer storage system that distributes files across multiple nodes instead of storing them in a single centralized server.

When the encrypted file is successfully uploaded to the IPFS network, the system generates a unique IPFS hash, also known as a content identifier (CID). This hash uniquely represents the file and acts as a permanent reference that can be used later to retrieve the file from the decentralized network.

Once the IPFS hash is generated, the system stores this hash in the blockchain network using a smart contract. Instead of storing the entire file on the blockchain, only the hash is recorded because blockchain storage is limited and expensive. The blockchain ensures immutability, meaning the stored hash cannot be modified or deleted after it is recorded. This feature guarantees data integrity, since any change in the file would produce a different hash that would not match the one stored on the blockchain.

In addition to blockchain storage, the system also saves file metadata in the MongoDB database. This metadata includes details such as the file name, user ID, upload date, IPFS hash, and blockchain transaction information. MongoDB acts as the main database that manages user accounts and file records.

7.5 COLLABORATIVE DIAGRAM

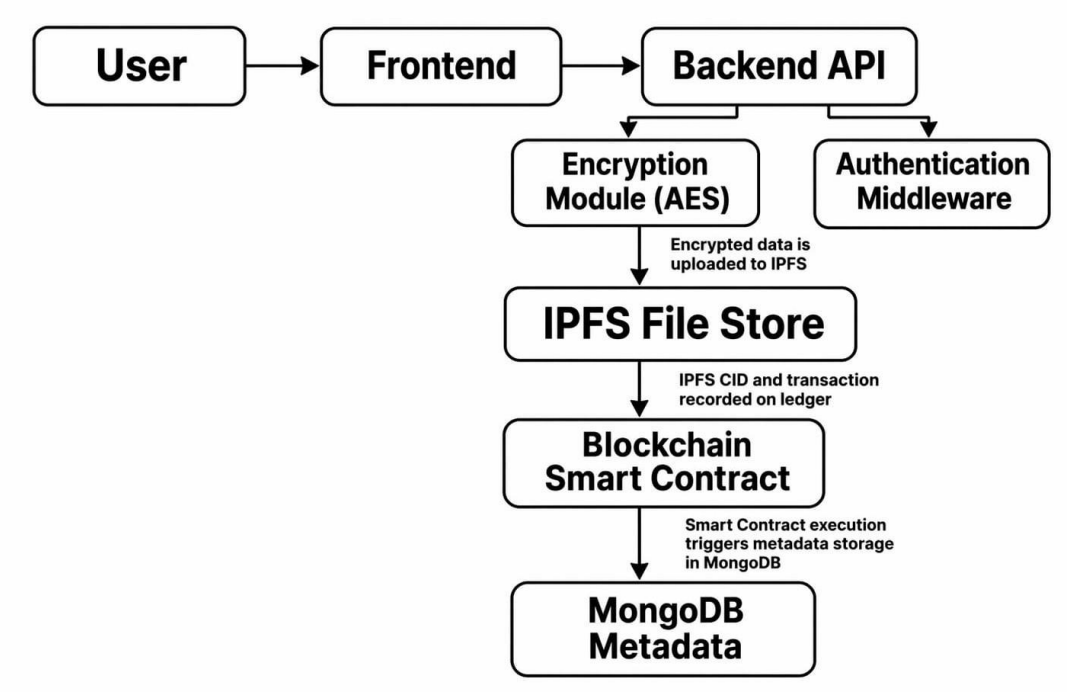


Figure 7.5 – Collaborative Diagram.

The collaborative diagram illustrates the high-level architecture of a secure data and metadata management system using IPFS and blockchain technology. The process begins with the User, who interacts with the system through the Frontend interface, typically a web application. The frontend sends user requests such as file upload, download, or verification to the Backend API, which handles the main application logic. Before processing any request, the backend user Authentication Middleware to verify the user's identity and ensure that only authorized users can access system resources. This authentication layer helps maintain system security and protects sensitive user data.After authentication, when a user uploads a file, the Encryption Module (AES**)** encrypts the file to ensure data confidentiality and protection from unauthorized access. The encrypted file is then uploaded to the IPFS (Interplanetary File System), which is a decentralized storage network. IPFS generates a unique Content Identifier (CID**)** or hash for each stored file.

8. IMPLEMENTATION

8.1 ENVIRONMENT SET:

The project is implemented using Node.js (or Python 3.8 or above) as the core programming language. The backend is developed using Express.js / Fast API to create REST APIs for handling user authentication, file upload, and blockchain interaction. The frontend is developed using HTML, CSS, and JavaScript (or react) to provide an interactive user interface. The blockchain environment is set up using Ethereum (Ganache) and smart contracts are written in Solidity. Web3.js (or Ethers.js) is used to connect the application with the blockchain network. Cloud storage such as AWS S3 (or local storage) is used to store files securely. The system is executed using VS Code as the development environment.

8.2 MODULE DESCRIPTION:

MODULES:

- User Authentication Module.
- File Management Module.
- Data Encryption & Hashing Module.
- Data Encryption & Hashing Module.
- Cloud Storage Module.
- Verification Module.
- Admin Monitoring & Control Module.

1. User Authentication Module:

The **User Authentication Module** is responsible for managing the registration and login processes of users within the system. During registration, users provide essential details such as **username, email address, and password**, which are securely stored in the system database.

2. File Management Module:

The File Management Module handles all operations related to file handling within the system. It manages the processes of uploading, downloading, viewing, and deleting files. When a user uploads a file, this module collects important file details such as file name, file size, file type, upload timestamp, and owner information. These details are stored as metadata in the database for easy tracking and management. This module interacts with the backend server and storage services to ensure that files are handled efficiently and securely. It also ensures proper file organization so that users can easily retrieve their files whenever required. The module maintains the relationship between users and their files, ensuring that only the file owner or authorized users can access or modify the stored files. Overall, the file management module ensures smooth and efficient file operations within the system.

3. Data Encryption & Hashing Module:

The Data Encryption and Hashing Module plays a critical role in protecting the confidentiality and integrity of stored files. Before a file is stored in the system, it undergoes an encryption process using secure algorithms such as AES (Advanced Encryption Standard). Encryption converts the original data into an unreadable format so that unauthorized users cannot access its contents. In addition to encryption, the module generates a cryptographic hash using the SHA-256 hashing algorithm. A hash is a fixed-length digital fingerprint of the file that uniquely represents its content. Even a small change in the file will result in a completely different hash value. This hash is used later for file verification and integrity checking. By combining encryption and hashing techniques, the system ensures both data confidentiality and tamper detection.

4. Blockchain Integration Module:

The Blockchain Integration Module connects the system with a blockchain network such as Ethereum. Instead of storing entire files on the blockchain, which would require large storage and high transaction costs, the system stores only the file hash generated by the hashing module. This hash is recorded through a smart contract.

5. Cloud Storage Module:

The Cloud Storage Module manages the storage of actual file data in a scalable and reliable storage environment. Files may be stored in decentralized storage systems such as IPFS (InterPlanetary File System) or cloud platforms like AWS S3. This module ensures that files are stored efficiently while maintaining high availability and reliability. When a file is uploaded, the encrypted version of the file is stored in the cloud storage system. The storage system generates a unique file reference or identifier, which is used to retrieve the file later. This module ensures that large files can be stored without burdening the blockchain network. By separating file storage from blockchain verification, the system achieves both efficient storage and secure verification.

6. Verification Module:

The Verification Module ensures that stored files remain authentic and unchanged over time. When a user requests to verify a file, the system generates a new hash of the current file and compares it with the original hash stored on the blockchain. If the two hash values match, it confirms that the file has not been modified since it was uploaded.However, if the hash values differ, the system identifies that the file has been altered or tampered with. In such cases, the module alerts the user about the integrity violation. This process provides strong assurance of file authenticity and data integrity, making the system reliable for applications where data trustworthiness is critical.

8.3 SAMPLE CODE

```
<! --fileController.js --
const ipfs = require("../utils/ipfs");
const File = require("../models/File");
const { contract, web3 } = require("../blockchain/eth");
// ================================
// UPLOAD FILE
// ================================
exports.uploadFile = async (req, res) => {
  try {
    if (!req.file) {
      return res.status(400).json({ message: "No file uploaded"
});


    }

    // upload to IPFS
    const result = await ipfs.add(req.file.buffer);
    const ipfsHash = result.path;

    // store hash on blockchain
    const accounts = await web3.eth.getAccounts();

    await contract.methods
      .storeFileHash(ipfsHash)
      .send({
        from: accounts[0],
        gas: 500000,
```

```
    });
   // save metadata in MongoDB
   const newFile = new File({
    ipfsHash: ipfsHash,
    fileName: req.file.originalname,
    originalName: req.file.originalname,
    mimeType: req.file.mimetype,
    size: req.file.size,
    owner: req.user.id,   // ✅ required
   });
   await newFile.save();
   res.json({
    message: "File uploaded successfully",
    file: newFile,
   });

  } catch (err) {
   console.error("UPLOAD ERROR:", err.message);

res.status(500).json({ message: err.message });
  }
};
// ================================
// LIST FILES
// ================================
exports.listFiles = async (req, res) => {
  try {
```

```
    const files = await File.find()



      .populate("owner", "email")
      .sort({ uploadedAt: -1 });

    res.json(files);
  } catch (err) {
    console.error("FETCH ERROR:", err);
    res.status(500).json({ message: "Error fetching files" });
  }

};
// ================================
// DOWNLOAD FILE
// ================================
const https = require("https");

const axios = require("axios");

// DOWNLOAD FILE
exports.downloadFile = async (req, res) => {
  try {
    const { hash } = req.params;

    // 🔷 Find file in MongoDB
    const file = await File.findOne({ ipfsHash: hash });
```

```
if (!file) {
    return res.status(404).json({ message: "File not found" });




 }
  // 🔷 Verify file hash on blockchain
  const isValid = await
contract.methods.verifyFileHash(hash).call();
  if (!isValid) {
    return res.status(400).json({ message: "File verification
failed" });
  }



  // 🔷 Fetch file from IPFS
  const ipfsURL = `https://dweb.link/ipfs/${hash}`;

  const response = await axios({
    method: "GET",
    url: ipfsURL,
    responseType: "stream",
  });

  // 🔷 Force file download with original filename
  res.setHeader(
    "Content-Disposition",
    `attachment; filename="${file.originalName}"`
  );
```

```
    // 🔷 Preserve original content type
    res.setHeader(
      "Content-Type",
      response.headers["content-type"] || "application/octet-
stream"


  // 🔷 Send file to browser
    response.data.pipe(res);


  } catch (err) {
    console.error("DOWNLOAD ERROR:", err.message);
    res.status(500).json({ message: "Download failed" });
  }
};

// ================================
// DELETE FILE
// ================================
exports.deleteFile = async (req, res) => {
  try {
    const { hash } = req.params;

    const file = await File.findOne({ ipfsHash: hash });

    if (!file) {
      return res.status(404).json({ message: "File not found" });
```

```
    }

    // only owner or admin can delete
    if (file.owner.toString() !== req.user.id) {


      return res.status(403).json({ message: "Not authorized" });


    }
    // unpin from IPFS (optional)
    try {


      await ipfs.pin.rm(hash);
      console.log("IPFS file unpinned:", hash);
    } catch {

    console.log("Unpin skipped");
    }
    await file.deleteOne();
    res.json({ message: "File deleted successfully" });
  } catch (err) {
    console.error("DELETE ERROR:", err);
    res.status(500).json({ message: "Delete failed" });
  }
};
<!-- eth.js -->
const Web3 = require("web3");
```

```
// load contract JSON
const contractData = require("./abi/FileStorage.json");

// connect to Ganache
const web3 = new Web3("http://127.0.0.1:7545");




// paste deployed contract address
const contractAddress =
"0x1813bF7732b5799Bc167e7bD3DE33E402E292Cd4";

// create contract

const contract = new web3.eth.Contract(
  contractData.abi,



  contractAddress
);
module.exports = { web3, contract };
<!-- filestorage.sol -->
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract FileStorage {
    mapping(string => bool) private storedHashes;
```

```
  event FileStored(string hash);
   function storeFileHash(string memory hash) public {
       storedHashes[hash] = true;
       emit FileStored(hash);
   }
   function verifyFileHash(string memory hash) public view
returns (bool) {
       return storedHashes[hash];
   }
}
<!-- dashboard.html --



<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,
initial-scale=1.0">

<title>Dashboard — BlockChain Storage</title>
<link
href="https://fonts.googleapis.com/css2?family=Inter:wght@3
00;400;500;600;700;800&display=swap" rel="stylesheet">
<style>



*{margin:0;padding:0;box-sizing:border-box;font-
family:'Inter',sans-serif;}

:root{
```

```
  --neon:#00d4ff;--neon2:#7c3aed;--bg:#020817;
  --card:rgba(255,255,255,0.04);--
border:rgba(255,255,255,0.08);
}

body{background:var(--bg);color:white;min-height:100vh;}

canvas{position:fixed;top:0;left:0;width:100%;height:100%;z-
index:0;pointer-events:none;}
.grid-bg{
  position:fixed;top:0;left:0;width:100%;height:100%;z-
index:0;
  background-image:

    linear-gradient(rgba(0,212,255,0.025) 1px,transparent 1px),

    linear-gradient(90deg,rgba(0,212,255,0.025)
1px,transparent 1px);
  background-size:60px 60px;pointer-events:none;
}

/* LAYOUT */
.layout{position:relative;z-index:1;display:flex;min-
height:100vh;}

/* SIDEBAR */
.sidebar{
  width:260px;min-height:100vh;flex-shrink:0;
  background:rgba(255,255,255,0.025);
  border-right:1px solid var(--border);
```

```
  backdrop-filter:blur(20px);



  padding:28px 16px;
  display:flex;flex-direction:column;gap:6px;
  position:fixed;top:0;left:0;height:100%;z-index:50;
}

.sidebar-logo{
  padding:8px 12px 24px;
  font-weight:800;font-size:18px;letter-spacing:-0.5px;
  border-bottom:1px solid var(--border);margin-bottom:10px;
}
.sidebar-logo span{background:linear-gradient(90deg,var(--
neon),var(--neon2));-webkit-background-
clip:text;background-clip:text;-webkit-text-fill-
color:transparent;}




.nav-item{
  display:flex;align-items:center;gap:12px;
  padding:11px 14px;border-radius:12px;
  font-size:14px;font-weight:500;color:rgba(255,255,255,0.5);
  cursor:pointer;transition:all 0.2s;
}

.nav-
item:hover{background:rgba(255,255,255,0.06);color:rgba(25
5,255,255,0.85);}
```

```
.nav-item.active{background:rgba(0,212,255,0.1);color:var(--neon);border:1px solid rgba(0,212,255,0.15);}
.nav-item .icon{font-size:18px;width:22px;text-align:center;}

.sidebar-footer{margin-top:auto;padding-top:16px;border-top:1px solid var(--border);}
.logout-btn{
  width:100%;display:flex;align-items:center;gap:12px;


  padding:11px 14px;border-radius:12px;
  font-size:14px;font-weight:500;color:rgba(239,68,68,0.7);
  cursor:pointer;transition:all 0.2s;background:none;border:none;
}
.logout-btn:hover{background:rgba(239,68,68,0.1);color:#f87171;}

/* MAIN */
.main{margin-left:260px;flex:1;padding:32px;}

/* TOPBAR */
.topbar{


  display:flex;align-items:center;justify-content:space-between;
  margin-bottom:28px;
}
.topbar h1{font-size:26px;font-weight:800;letter-spacing:-0.5px;}
```

```
.topbar-right{display:flex;align-items:center;gap:12px;}



.avatar{
  width:38px;height:38px;border-radius:10px;
  background:linear-gradient(135deg,var(--neon),var(--
neon2));
  display:flex;align-items:center;justify-content:center;
  font-weight:700;font-size:15px;
}

.stat{
  padding:20px;border-radius:16px;
  background:var(--card);border:1px solid var(--border);
  position:relative;overflow:hidden;transition:all 0.3s;
}
.stat:hover{transform:translateY(-3px);border-
color:rgba(0,212,255,0.2);}
.stat::before{
  content:'';position:absolute;top:0;left:0;right:0;height:2px;
```

9. SYSTEM TESTING

9.1 TYPES OF TESTINGS:

System testing ensures that the entire blockchain-based cloud storage system works as expected when all modules are integrated together. The following tests were performed during the implementation of the project.

9.2 TESTING OBJECTIVES:

- to verify the overall functionality of the system.
- to validate the integration of different technologies used in the system.
- to ensure data security and integrity.
- to verify system performance and reliability.
- to identify and eliminate errors or defects.

9.3 TYPES OF TESTS PERFORMED:

1. User Registration Test:

Objective:

To verify that new users can successfully register in the system.

Test Procedure:

- Open the registration page.
- Enter valid user details such as name, email, and password.
- Submit the registration form.

Expected Result:

The system stores user information in the **MongoDB.**

Actual Result:

User registration is successful, and the user record is created in the database.

2. User Login Test:

Objective:

To ensure that registered users can log in using valid credentials.

Test Procedure:

- Enter registered email and password.

Actual Result:

User is successfully authenticated and redirected to the dashboard.

3. File Upload Test:

Objective:

To verify that users can upload files successfully.

Test Procedure:

- Log in to the system.
- Select a file from the local system.
- Upload the file using the dashboard interface.

Expected Result:

The file should be received by the backend server and processed successfully.

Actual Result:

File is uploaded and processed without errors.

4. File Encryption Test:

- Upload a file.
- Verify encryption process before storage.

Expected Result:

The file content is encrypted using the **AES encryption algorithm** before sending it to IPFS.

Actual Result:

Files are encrypted successfully before storage.

8. IPFS Storage Test:

Objective:

To verify that encrypted files are stored in the IPFS decentralized storage network.

Test Procedure:

- Upload a file to the system.
- Monitor IPFS storage response**.**

Expected Result:

IPFS should generate a unique file hash (CID).

Actual Result:

File hash is generated successfully and returned to the backend.

9. Blockchain Hash Storage Test:

Objective:

To verify that the IPFS hash is correctly stored in the blockchain.

Test Procedure:

- Upload a file,
- Record the returned IPFS hash.
- Verify blockchain transaction.

Expected Result:

The IPFS hash should be stored in the **Ethereum smart contract**.

Actual Result:

Blockchain transaction is successful, and the hash is recorded.

10. File Metadata Storage Test:

Objective:

To ensure that file metadata is stored in the database.

Test Procedure:

- Upload a file.
- Check MongoDB database.

Expected Result:

Database should store:

- File name
- IPFS hash
- Upload timestamp
- User ID

Actual Result:

File metadata is stored successfully in MongoDB.

11. File Download Test:

Objective:

To verify that stored files can be retrieved.

Test Procedure:

- Select a previously uploaded file.
- Request download.

Expected Result:

System retrieves the file from IPFS using the stored hash and sends it to the user.

Actual Result:

File is successfully retrieved and downloaded.

9.4 TEST CASES

Here ae the few TEST CASES to ensures that the entire blockchain-based cloud storage system works as expected when all modules are integrated together.

Test Case ID	Test Case Description	Input	Expected Output	Actual Result	Status
TC01	User Registration	Enter valid name, email, and pasword	User account should be created in MongoDB	User registered successfully	Pass
TC02	Duplicate User Registration	Enter an already registered email	System should display "User already exists" error	Error message displayed	Pass
TC03	User Login (Valid Credentials)	Enter correct email and password	User should be redirected to dashboard	Login successful	Pass
TC04	User Login (Invalid Credentials)	Enter wrong password	System should show authentication error	Error message displayed	Pass
TC05	File Upload	Upload a valid file	File should be accepted processed by backend	File uploaded successfully	Pass
TC06	File Encryption	Upload file and verify encryption pr	File should be encrypted before storing	File encrypted succesfully	Pass
TC07	IPFS Storage	Upload encrypted file	IPFS hash generated	IPFS hash generated	Pass
TC08	Blockchain Hash Storage	Store IPFS hash on blockchain	Hash should be recorded in smart contract	Transaction successful	Pass
TC09	Metadata Storage	Upload file	File downloaded successfully ard	Metadata saved successfully	Pass
TC10	File Download store vash token	Download file store hash ard store hash	File downloaded successfully and hash verified	File downloaded successfully and hash verified	Pass
TC11	Unauthorized Access	Attempt access secure REST API without token	File downloaded successfully and hash verified	Access denied message displayed	Pass
TC12	Unauthorized Access	System should deny acsss	System should deny access	Access denied message	Pass

Table 9.4 Test Cases.

10. RESULT SCREENSHOTS

10.1 REGISTRATION PAGE

The image shows a clean registration page for a blockchain-based application running on localhost. It includes fields for name, email, and password, along with a “Create Secure Account” button. The design emphasizes security with a vault icon and also provides a login link for existing users.

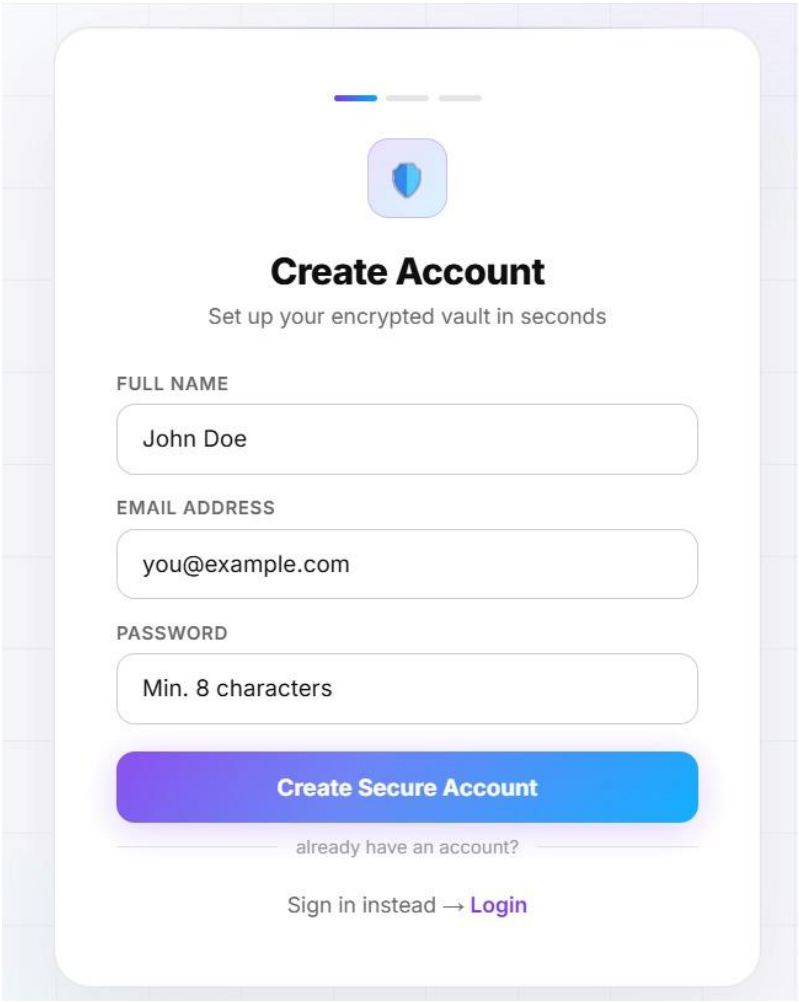


Figure 10.1 – Registration Page.

10.2 LOGIN PAGE

The image shows a secure login page for a blockchain-based application running on localhost. It includes fields for email, password, and a 6-digit OTP for two-factor authentication, along with a “Sign In Securely” button. The design highlights security and also provides options to reset a password or create a new account.

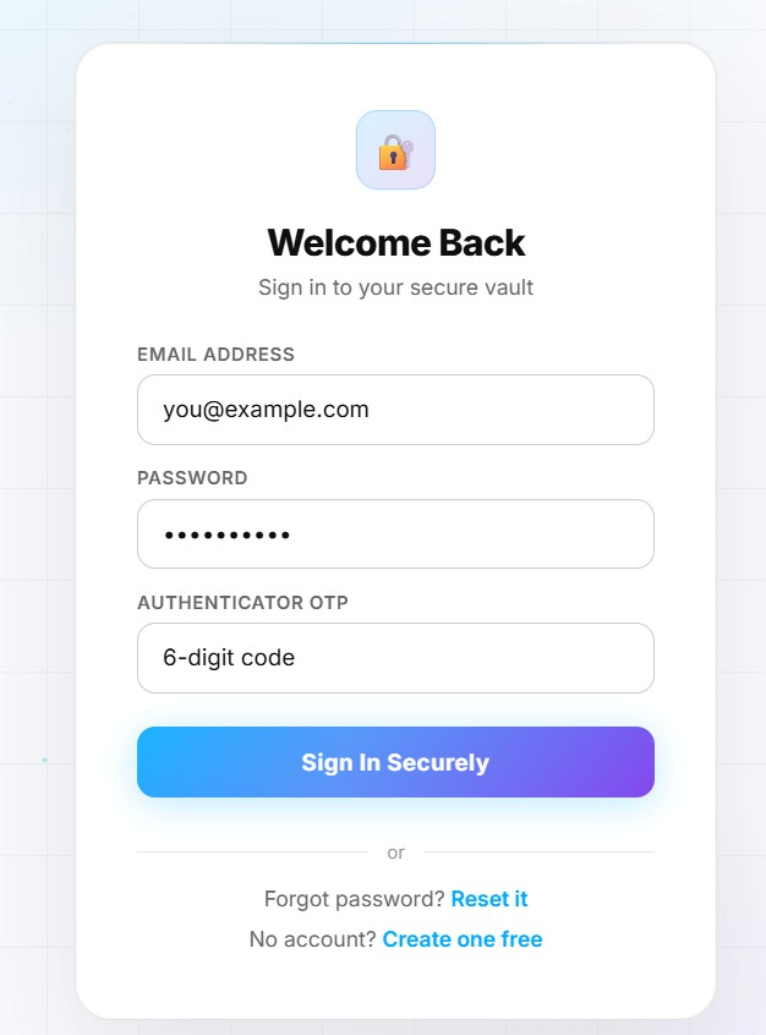


Figure 10.2 – Login Page.

10.3 DASHBOARD

This interface shows a blockchain-based storage dashboard (“My Vault”) where users can manage encrypted files and track stats like total files, downloads left, and encryption status (AES, on-chain recorded).

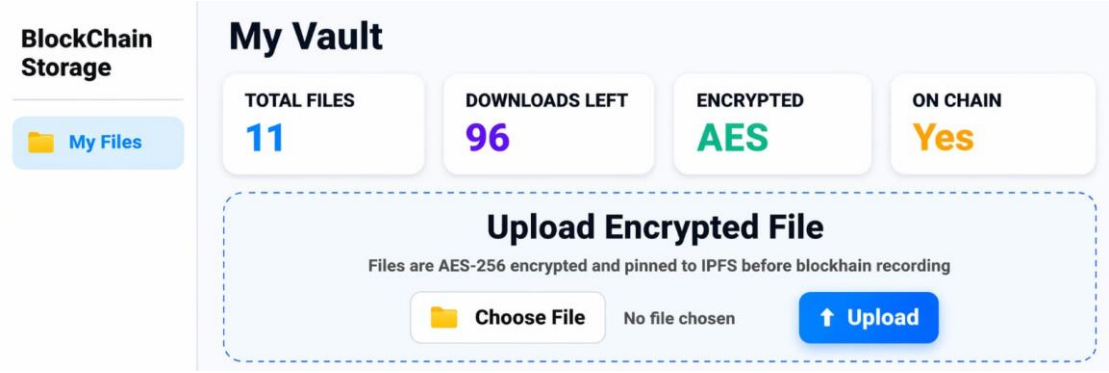
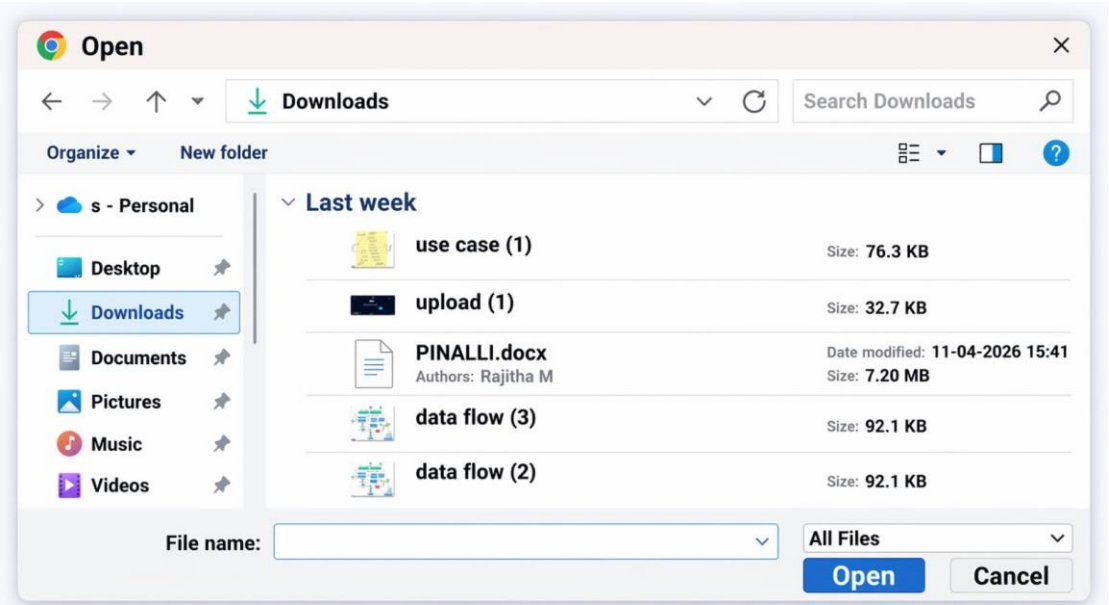


Figure 10.3 – Dashboard.

10.4 FILE UPLOAD

This is a file selection window showing the Downloads folder, where the user can browse and choose a file to upload or open.It displays recent files with details like size and date, and provides options to open the selected file or cancel the action.



. Figure 10.4- File Upload.

10.5 PREVIEW FILE UPLOAD

This screen shows the file upload section where a selected file (“use case.jpeg”) is ready to be uploaded securely.The file will be encrypted using AES-256 and stored on IPFS before being recorded on the blockchain.

Figure 10.5 – Preview File Upload.

10.6 UPLOAD FILE SUCCESSFUL

This screen shows the upload section where no file is currently selected, but the system confirms that a file has been successfully encrypted and stored on IPFS.It indicates that the upload and encryption process was completed securely before blockchain recording.

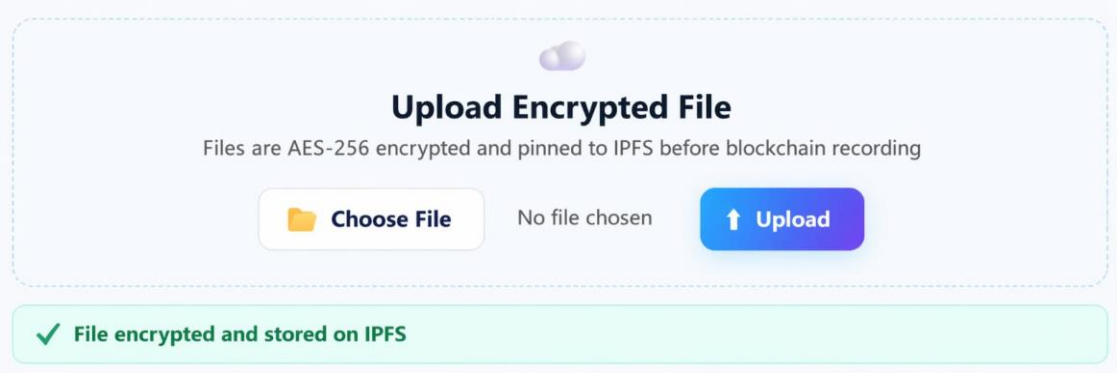


Figure 10.6 – Upload File Successful.

10.7 FILE PREVIEW

The image shows a file preview popup on the blockchain storage dashboard, displaying a diagram of a messaging system architecture. The file will be encrypted using AES-256 and stored on IPFS before being recorded on the blockchain It illustrates components like HTTP/WebSocket gateways, chat services, message synchronization, and notification systems, explaining how messages flow between users and backend services.

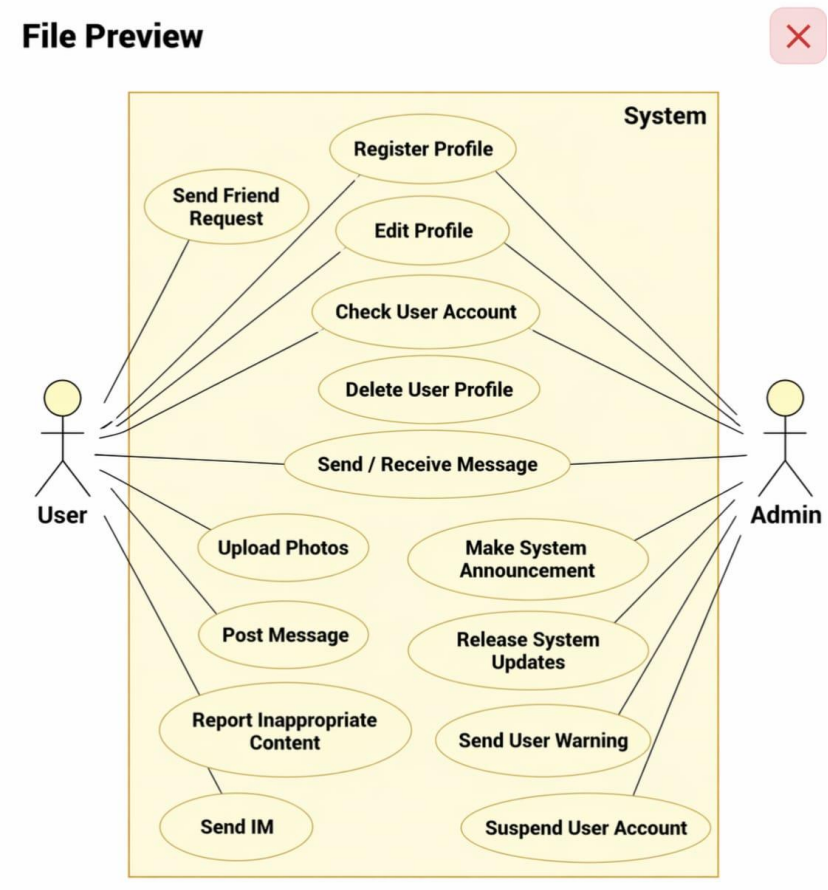


Figure 10.7 File Preview.

10.8 FILE DOWNLOAD

This notification shows that the file “use case (5).jpeg” has been successfully uploaded or processed.The “Done” status confirms the operation is complete without errors.

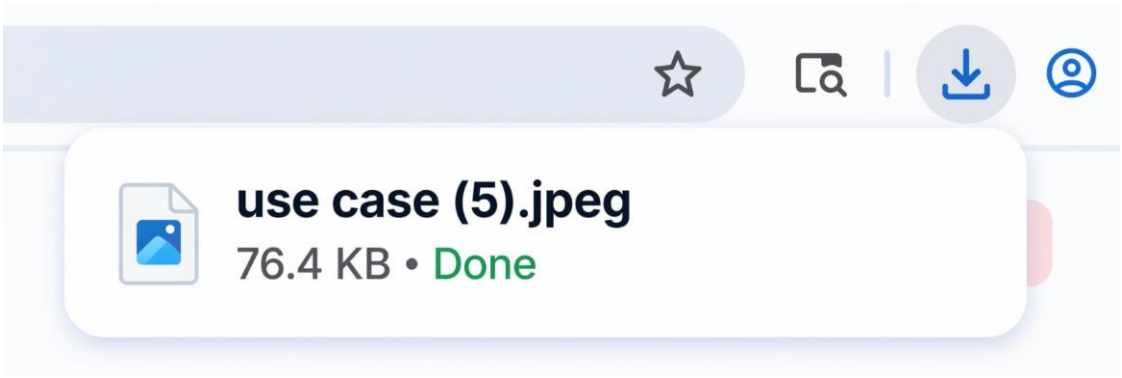


Figure 10.8 – File Download.

10.9 FILE DELETE

When an user or the client what to delete permanently from the cloud an webserver’s windows pops up for conforming to delete the file permanently from the IPFS.

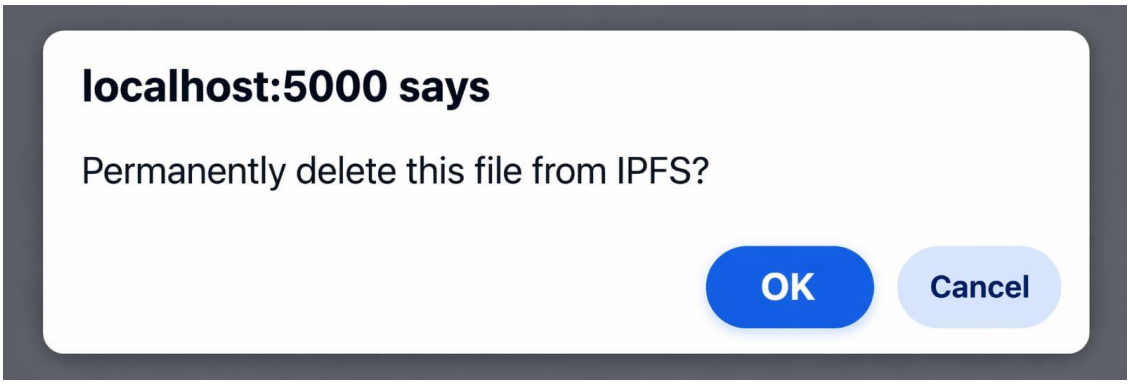


Figure 10.9 – File Delete.

11. CONCLUSION

Combining cloud storage with blockchain technology creates a secure, transparent, and efficient framework for managing digital files. In this system, files are first fragmented into smaller pieces and encrypted before being stored across multiple cloud storage nodes. This distribution ensures that even if one storage node is compromised, the complete file cannot be accessed or reconstructed without proper authorization. Encryption further strengthens security by protecting the file content from unauthorized access. By spreading encrypted data across multiple nodes, the system increases both reliability and resilience compared to traditional centralized storage systems.

Blockchain technology adds another layer of protection by maintaining an immutable and transparent record of file-related activities. Instead of storing entire files on the blockchain, the system stores file hashes and transaction details in a distributed ledger. These records include information about file ownership, access permissions, and verification data. Because blockchain data cannot be modified once it is recorded, it ensures that the file integrity remains intact and prevents unauthorized modifications. This transparency also allows users to verify file authenticity at any time, creating a trustworthy environment for managing sensitive information.

Another key component of the system is the use of smart contracts, which automate several important operations within the platform. Smart contracts handle tasks such as user authentication, file verification, and access control without requiring manual intervention. By automating these processes, the system reduces the chances of human error and improves operational efficiency. Smart contracts also ensure that access permissions are strictly enforced, allowing only authorized users to interact with stored files. This automated management improves the reliability and security of the entire system.

12. FUTURE SCOPE

The integration of blockchain technology with cloud storage is transforming the way digital data is stored, managed, and controlled. In the traditional cloud storage model, users rely heavily on a few large service providers who maintain centralized data centres. This centralized structure often creates a single point of failure, meaning that a security breach, technical failure, or change in service policies can place a vast amount of data at risk. Organizations and individuals are essentially "renting" storage space, and their control over the data is limited by the policies and infrastructure of the provider. By integrating blockchain with cloud storage systems, a new model of decentralized data sovereignty is emerging, where users have greater control over their data and its accessibility.

In a decentralized storage environment powered by blockchain, data is no longer stored in a single location or data centre. Instead, files are divided into smaller fragments, encrypted for security, and distributed across multiple independent nodes around the world. This distribution ensures that no single entity has complete control over the stored data. Even if one node fails or is compromised, the system can still reconstruct the file from the remaining fragments stored across the network. This approach significantly improves data availability, reliability, and resilience against cyberattacks. At the same time, blockchain technology records important information about the file—such as its ownership, access permissions, and verification hashes—creating a transparent and tamper-proof record of all data-related transactions. Another important component of this decentralized ecosystem is the use of smart contracts, which function as automated digital agreements stored on the blockchain. Smart contracts can act as "digital notaries," verifying that a file exists, confirming that it has not been altered, and ensuring that only authorized users can access it. These contracts automatically execute predefined rules without requiring human intervention or centralized authorities.

13. REFERENCES

1. Varshini, M., Chandravathi, M., Mani Rekha, G., Balaraju, M., Afraz, M., Sarvanan, M., & Dharnasi, P. (2026). ATM access using card scanner and face recognition with AIML. International Journal of Research Publications in Engineering, Technology and Management (IJRPETM), 9(1), 113–118.

2. Inbavalli, M., & Arasu, T. (2015). Efficient analysis of frequent item set association rule mining methods. International Journal of Scientific & Engineering Research, 6(4).

3.Sugumar, R. (2025). Explainable AI-driven secure multi-modal analytics for financial fraud detection and cyber enabled pharmaceutical network analysis. International Journal of Advanced Research in Computer Science & Technology (IJARCST), 8(6), 13239–13249.

4.Poornima, G., & Anand, L. (2025). Medical image fusion model using CT and MRI images based on dual scale weighted fusion based residual attention network with encoder–decoder architecture. Biomedical Signal Processing and Control, 108, 107932.

5.Prasanna, D., & Manishvarma, R. (2025, February). Skin cancer detection using image classification in deep learning. In 2025 3rd International Conference on Integrated Circuits and Communication Systems (ICICACS) (pp. 1–8). IEEE.

6.Sammy, F., Chettier, T., Boyina, V., Shingne, H., Saluja, K., Mali, M., ... & Shobana, A. (2025). Deep learning driven visual analytics framework for next-generation environmental monitoring. Journal of Applied Science and Technology Trends, 114–122.

7.Lakshmi, A. J., Dasari, R., Chilukuri, M., Tirumani, Y., Praveena, H. D., & Kumar, A. P. (2023, May). Design and implementation of a smart electric fence built on solar with an automatic irrigation system. In 2023 2nd International Conference on Applied Artificial Intelligence and Computing (ICAAIC) (pp. 1553–1558). IEEE.